

Christof Teuscher

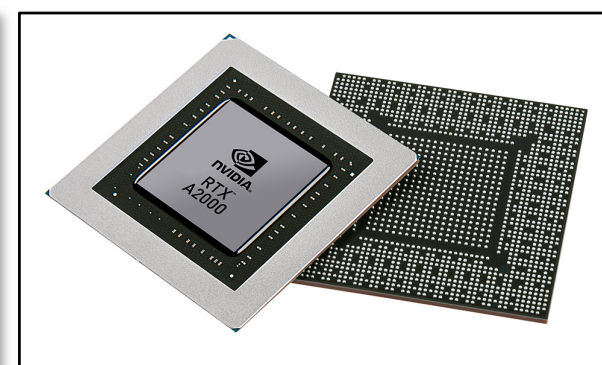
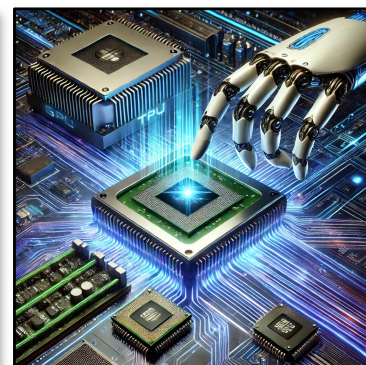
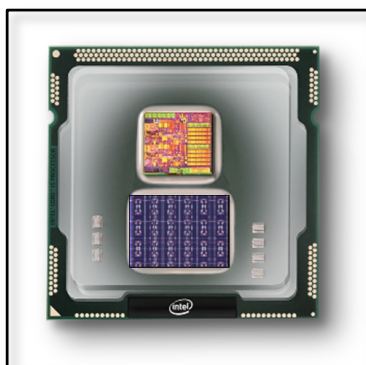
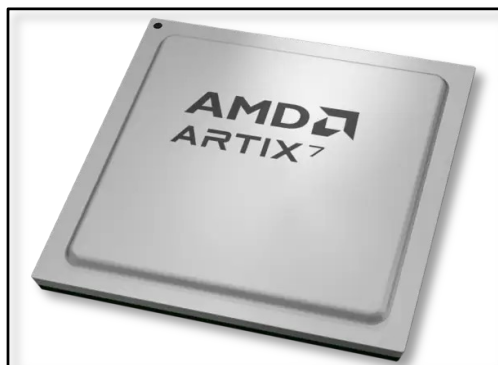
ECE 410/510: Hardware for AI and ML, Spring 2026

Hardware for AI and ML Overview + Co-design

Portland State University
Department of Electrical and Computer Engineering (ECE)

www.teuscher-lab.com

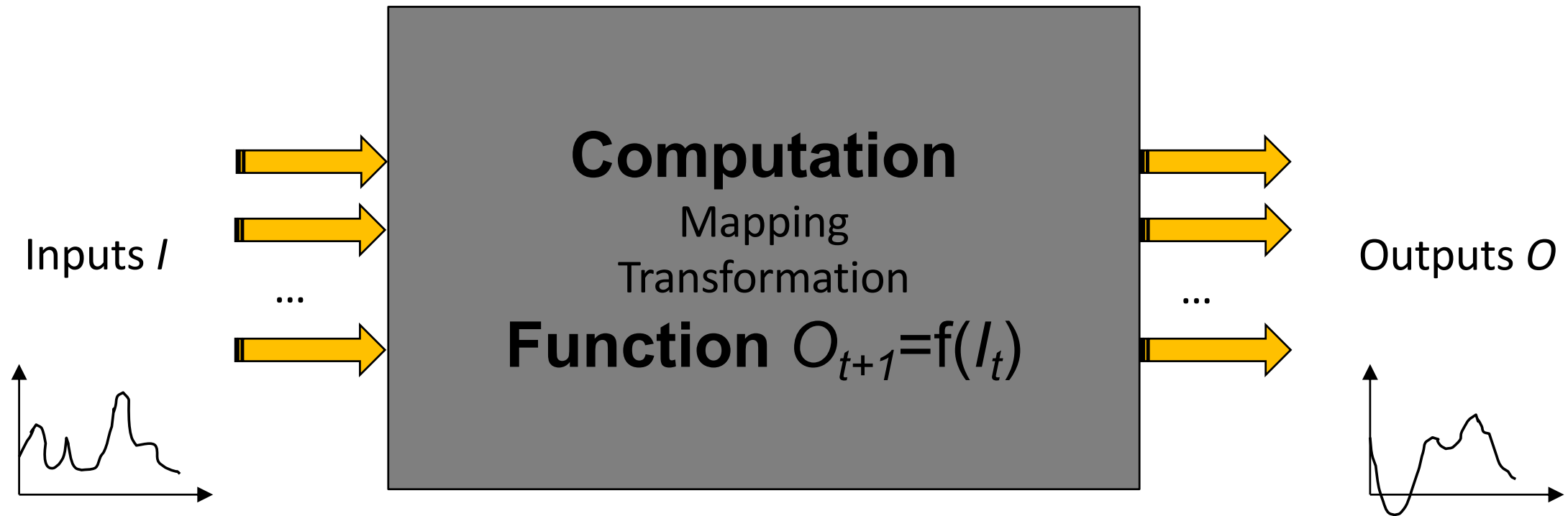
teuscher@pdx.edu



Today

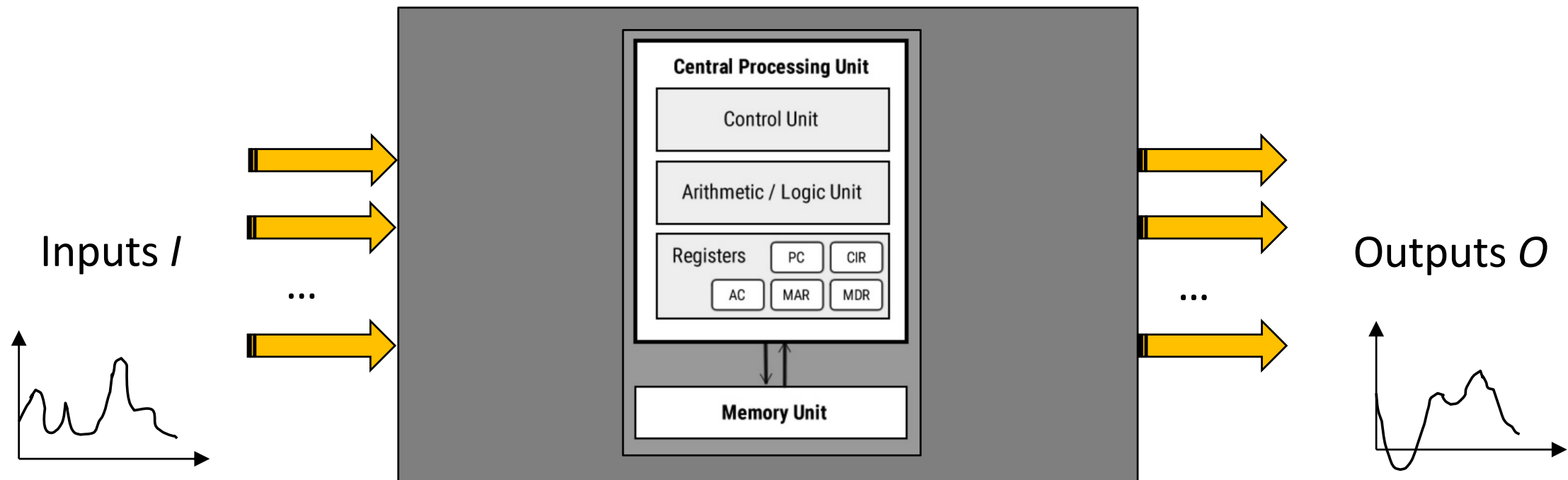
- What's the job of a computer architect / chip designer?
- What are the trade-offs?
- Why not do everything in software?
- Where do I draw the HW/SW boundary?
- What HW do I build?
- How do I analyze an algorithm/workload?
- How do I map an algorithm onto HW.
- ...

What is computation?

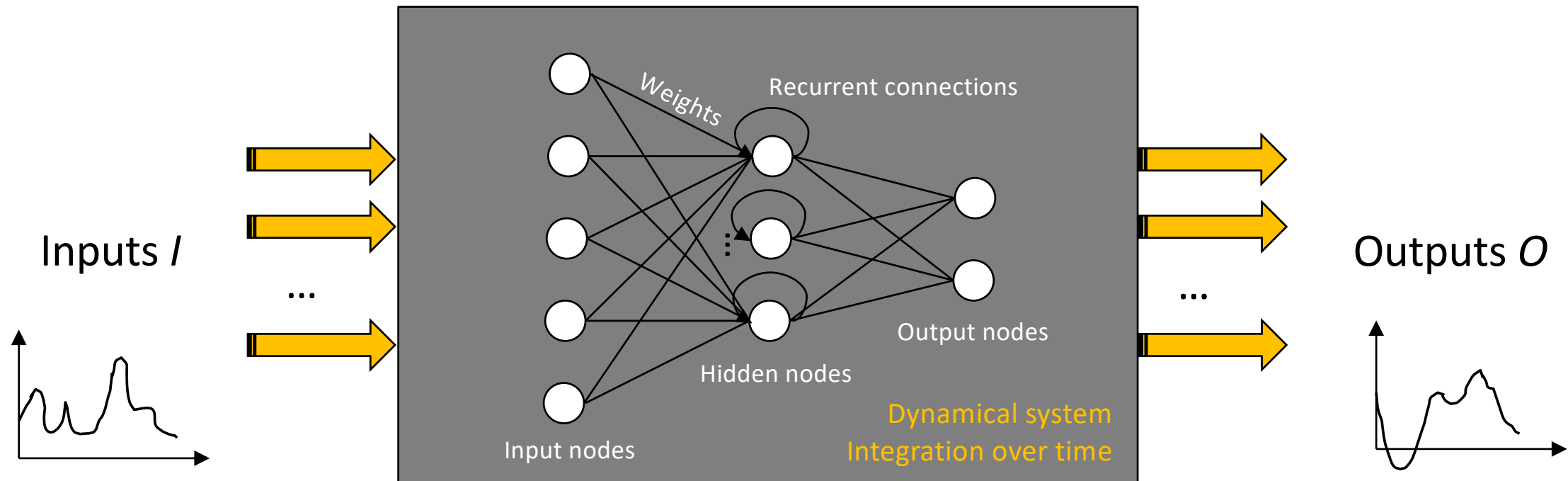


How can we “realize” this mapping?

What is computation?



What is computation?



Recurrent neural networks (RNN)

Training is challenging, various algorithms exist (backpropagation through time), convergence not guaranteed, many weights, slow learning. RNNs are also universal function approximators

(Schaefer & Zimmerman, 2007, <https://doi.org/10.1142/S0129065707001111>)

Special types of neural networks

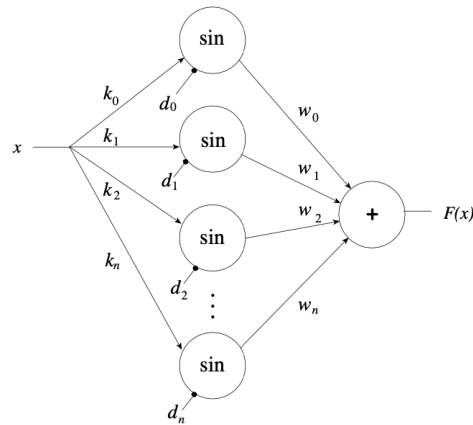


Fig. 1.17. A Fourier network

Figure 1.17 shows how a Fourier series can be implemented as a neural network. If the function F is to be developed as a Fourier series it has the form

$$F(x) = \sum_{i=0}^{\infty} (a_i \cos(ix) + b_i \sin(ix)). \quad (1.2)$$

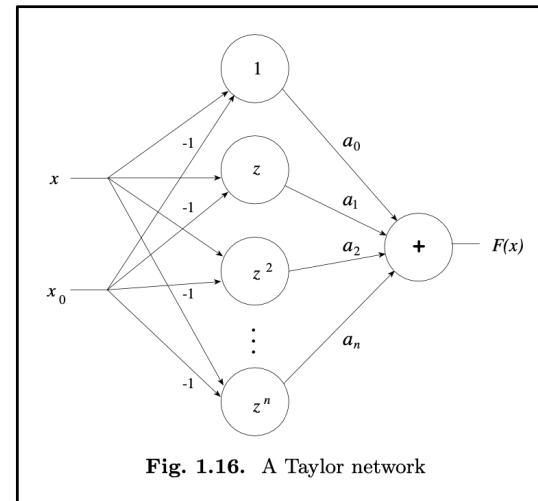
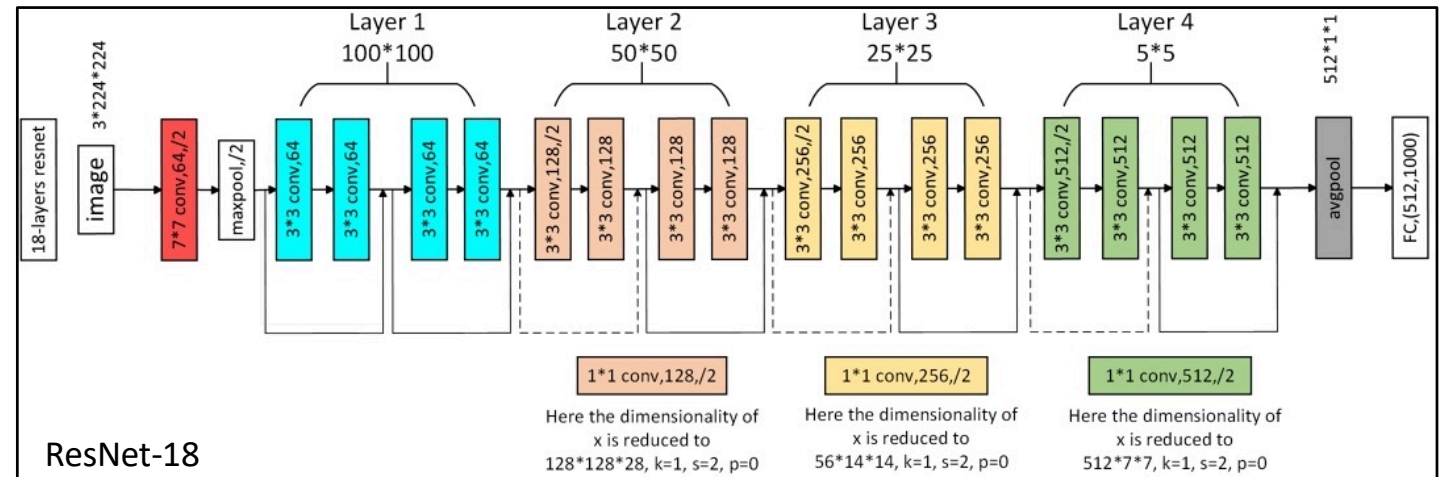
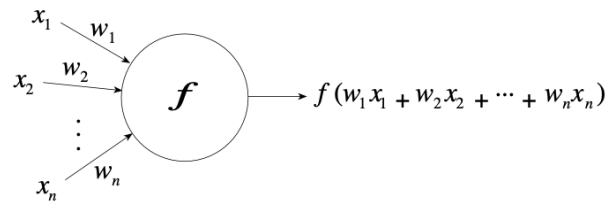


Fig. 1.16. A Taylor network

$$F(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2 + \dots + a_n(x - x_0)^n + \dots,$$



Neural networks as matrix operations



Scalar (single neuron, n inputs)

net input

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

which in vector dot-product notation is

$$z = \mathbf{w}^T \mathbf{x} + b$$

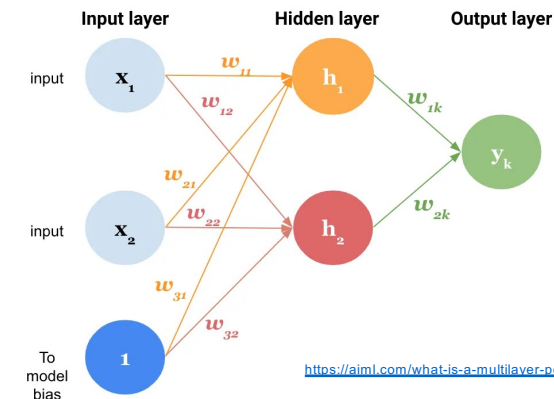
$\mathbf{w} \in \mathbb{R}^n$ (weights)

$\mathbf{x} \in \mathbb{R}^n$ (inputs)

$b \in \mathbb{R}$ (bias)

output (after activation)

$$a = \varphi(z) = \varphi(\mathbf{w}^T \mathbf{x} + b)$$



Matrix form (layer of m neurons, n inputs)

pre-activations for all m neurons at once

$$z = \mathbf{W}\mathbf{x} + \mathbf{b}$$

$\mathbf{z} \in \mathbb{R}^m$

$\mathbf{W} \in \mathbb{R}^{m \times n}$

$\mathbf{x} \in \mathbb{R}^n$

$\mathbf{b} \in \mathbb{R}^m$

$$\mathbf{a} = \varphi(\mathbf{W}\mathbf{x} + \mathbf{b}) \text{ (}\varphi \text{ applied element-wise)}$$

The key insight: a whole layer of neurons is just one **matrix-vector multiply**, which is why GPUs (optimized for exactly that operation) accelerate neural nets so effectively.



Christof Teuscher



teuscher@pdx.edu

www.teuscher-lab.com/teaching



What's the relationship between a dot product and MAC?

?

What's the relationship between a dot product and MAC?

- A single MAC computes:

$$\text{accumulator} \leftarrow \text{accumulator} + (a_i \times b_i)$$

- A dot product of length n is just n MACs chained together:

$$\mathbf{w}^T \mathbf{x} = \sum_i w_i x_i = \text{MAC}_1 + \text{MAC}_2 + \cdots + \text{MAC}_n$$

- So, the relationship is:

- One MAC = one multiply + one add (2 FLOPs)
- One dot product of length n = n MACs = $2n$ FLOPs
- One neuron's output = one dot product + bias add + activation $\approx 2n$ FLOPs
- One layer (m neurons, n inputs) = m dot products = $2mn$ FLOPs
- This is why people count layer cost as $2mn$ MACs.

What's the most common operation in neural nets?

General Matrix-Matrix Multiply (GEMM)

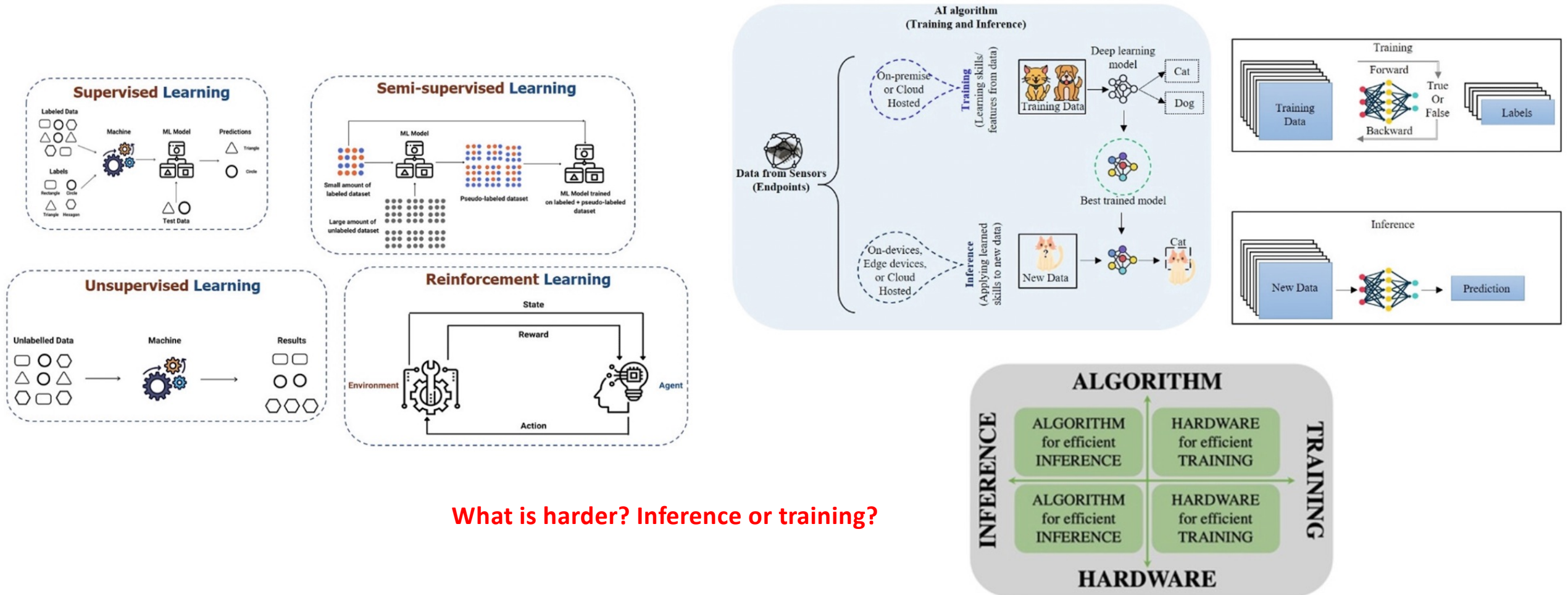
$$C = \alpha AB + \beta C$$

$\alpha = 1$ and $\beta = 0$ it reduces to a plain matrix multiply $C = AB$.

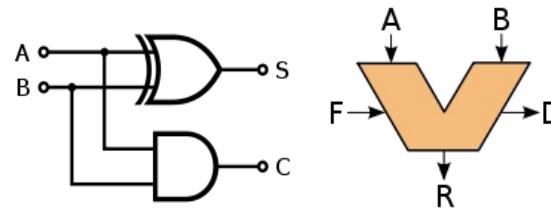
- When you process a batch of inputs rather than a single vector, the layer equation becomes:
$$\mathbf{Z} = \mathbf{W} \mathbf{X} + \mathbf{B}$$

where now $\mathbf{X} \in \mathbb{R}^{n \times b}$ (n inputs, b samples), $\mathbf{Z} \in \mathbb{R}^{m \times b}$. **That's just one GEMM call.**
- Virtually everything in a neural network reduces to it:
 - Fully connected layers**
 - Convolutions**
 - Attention** (transformers): Q, K, V projections are GEMM; the attention score matrix QK^T is GEMM; the weighted sum AV is GEMM
 - Embeddings lookup**: a degenerate GEMM with a one-hot matrix
 - Recurrent layers** (LSTM, GRU): the gate computations are all GEMM
- This is why hardware like GPUs and Tensor Processing Units (TPUs) are essentially giant systolic arrays purpose-built to stream GEMM as fast as physics allows.
- When someone says a GPU is "good at deep learning," what they really mean is it's good at GEMM. Everything else follows from that.**

Learning

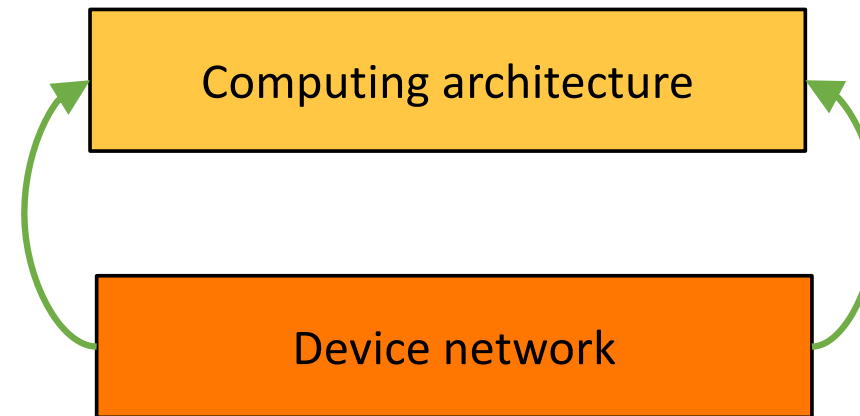


Intrinsic vs designed computation

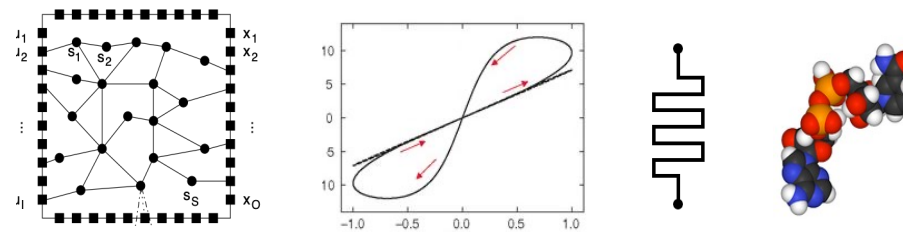


Desired computation

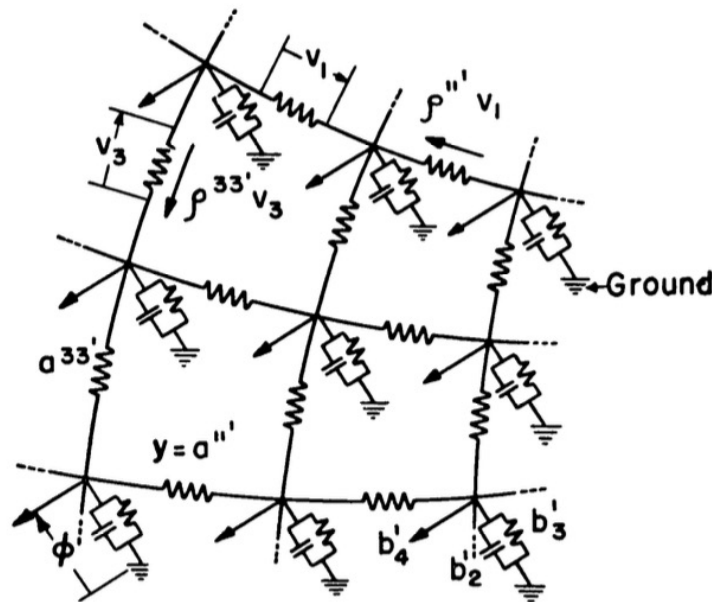
Intrinsic dynamics



How to obtain a desired computation?



Intrinsic vs designed computation



VS

$$\phi = b_2 \frac{\partial \phi}{\partial t} + b_3 \phi + b_4$$

Kron, Gabriel. "Electric circuit models of partial differential equations." Electrical Engineering 67, no. 7 (1948): 672-684.

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters
L = 1.0 # Length of the domain
T = 0.1 # Total time
nx = 50 # Number of points in space
nt = 500 # Number of points in time
alpha = 0.1 # Thermal diffusivity

# Grid spacing
dx = L / (nx - 1)
dt = T / (nt - 1)

# Initialize solution array
u = np.zeros((nt, nx))

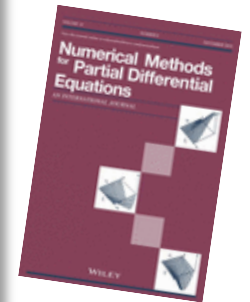
# Initial condition
u[0, :] = np.sin(np.pi * np.linspace(0, 1, nx))

# Boundary conditions
u[:, 0] = 0
u[:, -1] = 0

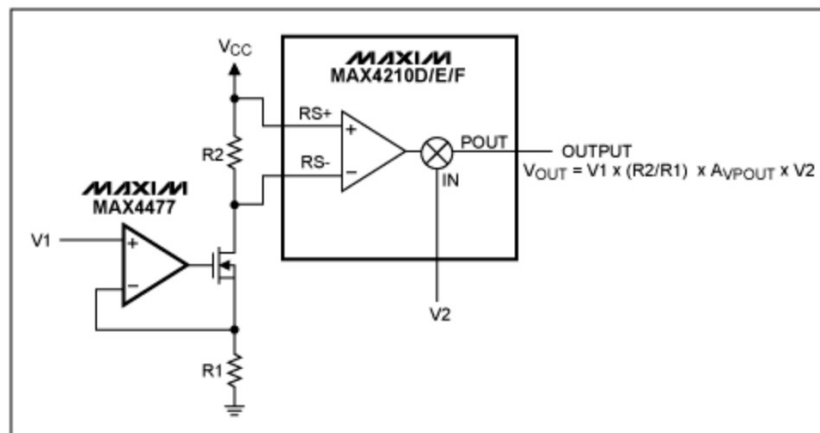
# Finite difference loop (explicit method)
for n in range(0, nt - 1):
    for i in range(1, nx - 1):
        u[n + 1, i] = u[n, i] + alpha * dt / dx**2 * (u[n, i + 1] - 2 *

# Plotting the results
x = np.linspace(0, L, nx)
t = np.linspace(0, T, nt)

plt.figure()
plt.imshow(u, extent=[0, L, T, 0], aspect='auto', cmap='viridis')
plt.xlabel('x')
plt.ylabel('t')
plt.title('Heat Equation Solution')
plt.colorbar(label='Temperature')
plt.show()
```



Intrinsic vs designed computation



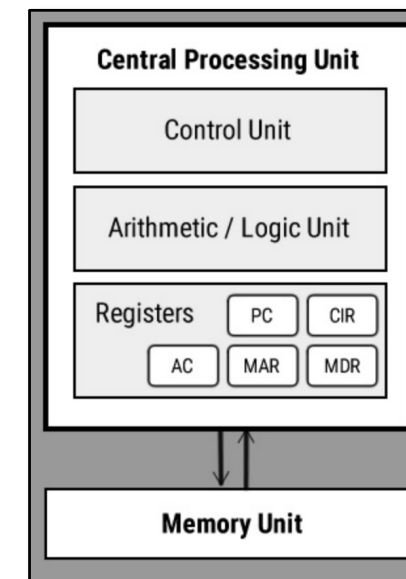
VS

```
num1 = 10
num2 = 5
product = num1 * num2
print(product) # Output: 50

float1 = 3.14
float2 = 2.0
float_product = float1 * float2
print(float_product) # Output: 6.28
```

Figure 1. A generic 1V analog multiplier using the MAX4210D/E/F and MAX4477.

<https://www.analog.com/en/resources/design-notes/using-a-generic-0-to-1v-analog-multiplier-to-ensure-accurate-power-management.html>





Christof Teuscher



teuscher@pdx.edu

www.teuscher-lab.com/teaching



What are the design trade-offs?

?

What are the design trade-offs?

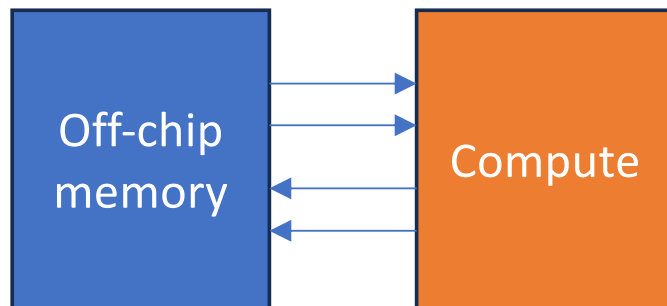
- **Performance**
- **Flexibility** (general purpose vs application-specific)
- **Energy efficiency**
- **Power density** (avoid burning chip, cost of cooling technology)
- **Design complexity**
- **Design cost**
- **Scalability**
- **Shrinking design schedules** (time-to-market)

- [PPAC: Performance, Power, Area, Cost]

Why design something new?

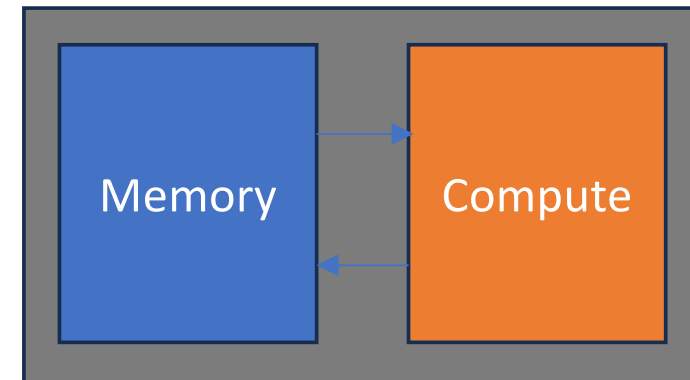
Problem #1: Data locality

Traditional architecture



High latency
High energy

Efficient AI architecture

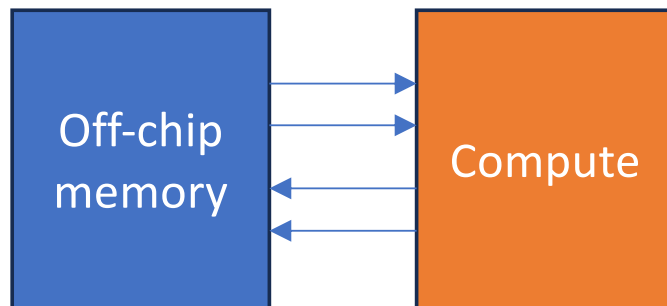


Data locality

Why design something new?

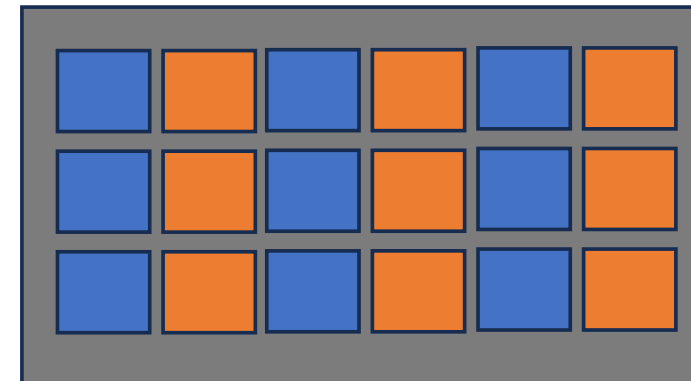
Problem #2: The need for parallelism

Traditional architecture



High latency
High energy

Efficient AI architecture

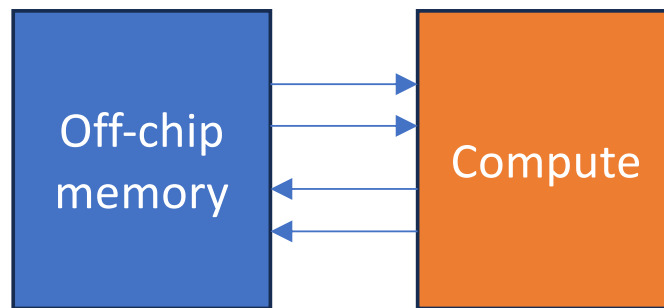


Massive parallelism

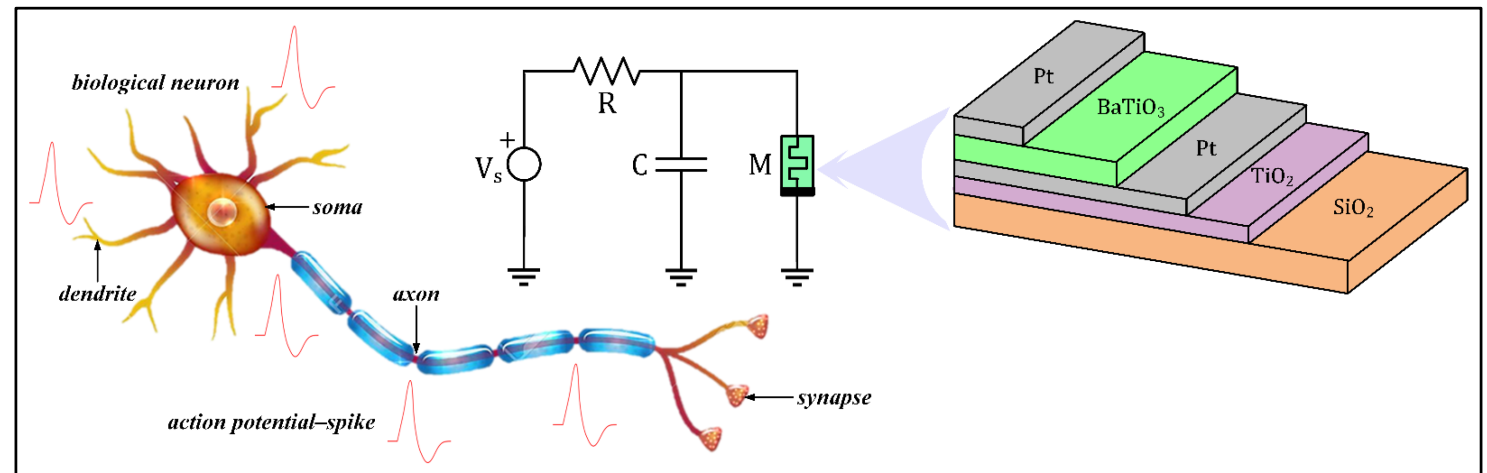
Why design something new?

Problem #3: Compute closer to the physics

Traditional architecture



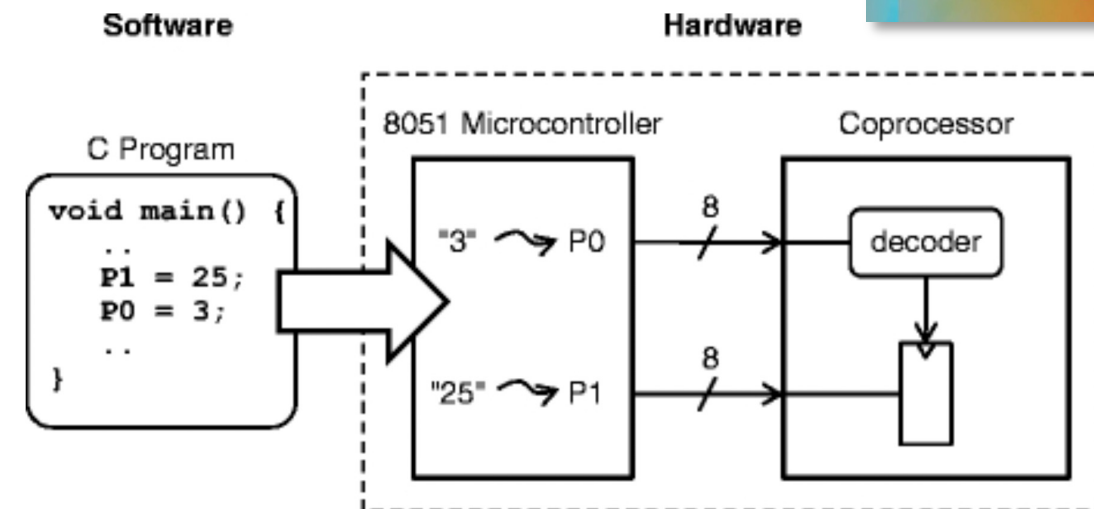
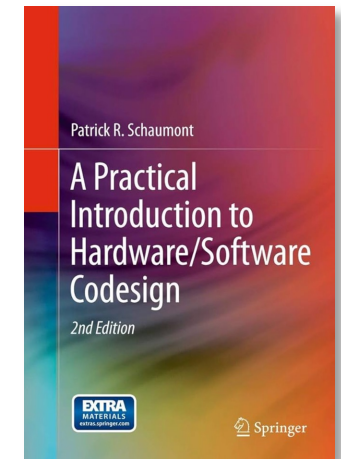
High latency
High energy



<https://www.mdpi.com/2079-9292/11/6/894>

What is (HW/SW) co-design?

- The practice of taking the **best from software** design and the **best from hardware** design to solve design problems.
- HW/SW co-design deals with HW/SW **interfaces**.
- HW/SW co-design is the design of cooperating HW and SW components in a **single design effort**.
- Why design new HW if there is already one available to do the job? E.g., a RISC processor?

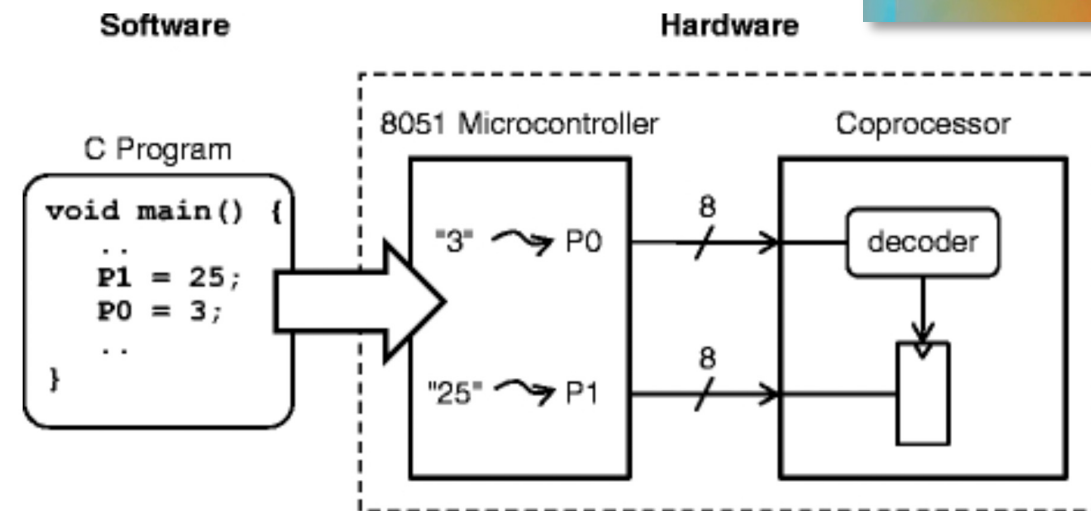
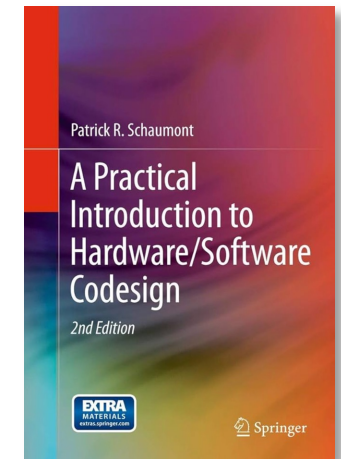


Schaumont, 2013

What is (HW/SW) co-design?

- The practice of taking the **best from software** design and the **best from hardware** design to solve design problems.
- HW/SW co-design deals with HW/SW **interfaces**.
- HW/SW co-design is the design of cooperating HW and SW components in a **single design effort**.

- Why design new HW if there is already one available to do the job? E.g., a RISC processor?
- From a flexibility and design effort perspective, as much as possible should be in software. [Easy, flexible, available libraries, etc.]



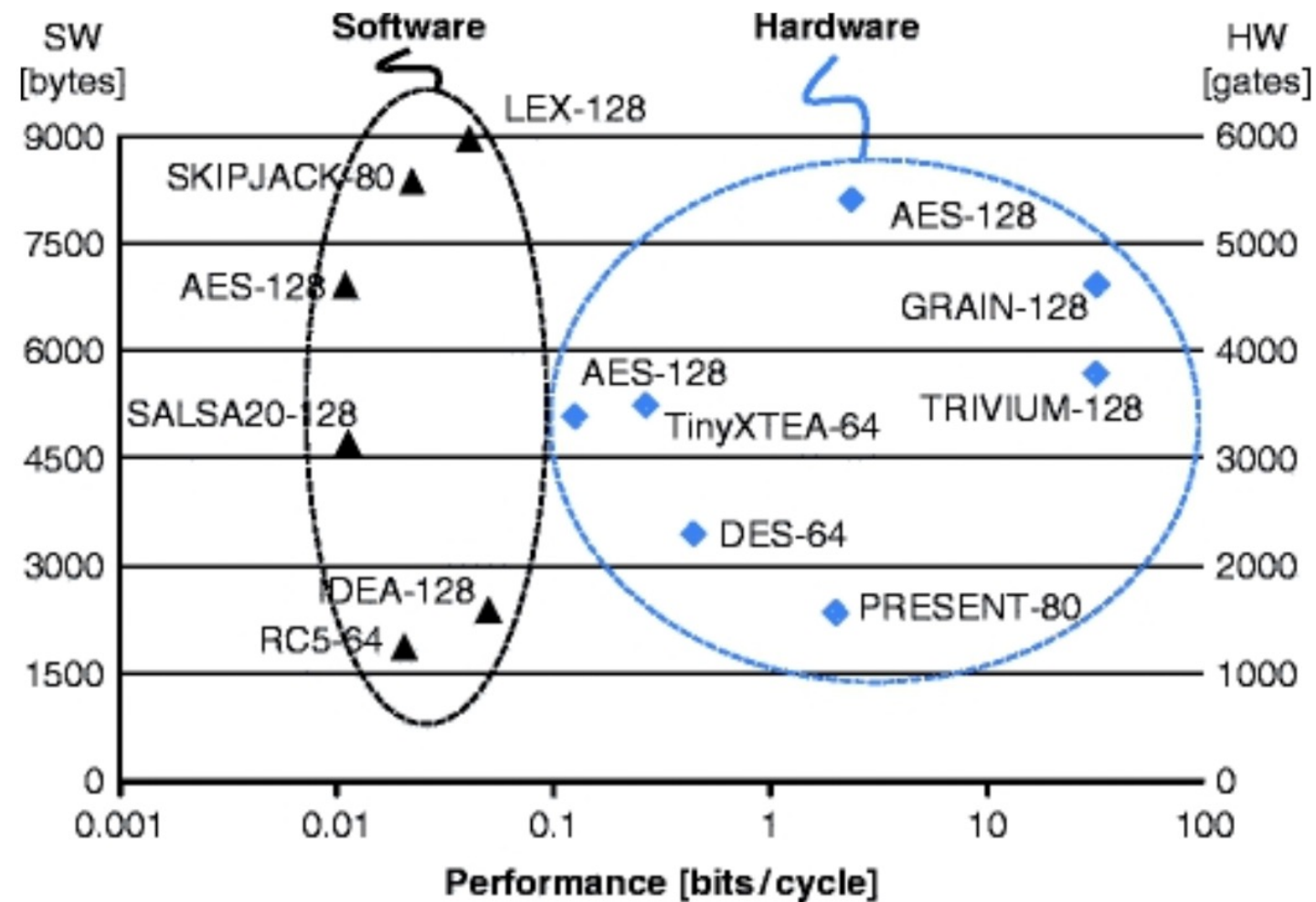
Schaumont, 2013

Why is HW/SW co-design important for AI/ML

- **Performance optimization:** Traditional hardware design assumes fixed software, while traditional software design assumes fixed hardware. Co-design breaks this barrier by simultaneously optimizing both, leading to significant performance improvements for AI workloads.
- **Specialized acceleration:** AI algorithms have unique computation patterns (like matrix multiplications and convolutions) that benefit from custom hardware accelerators tailored to these specific operations.
- **Memory bottlenecks:** AI models face severe memory bandwidth constraints. Co-designing hardware memory hierarchies with software that efficiently schedules operations can minimize data movement and maximize throughput.
- **Energy efficiency:** By tailoring hardware precisely to the needs of ML workloads and optimizing software to take advantage of hardware capabilities, co-design dramatically reduces power consumption, which is critical for both data centers and edge devices.

Cryptography on small embedded platforms

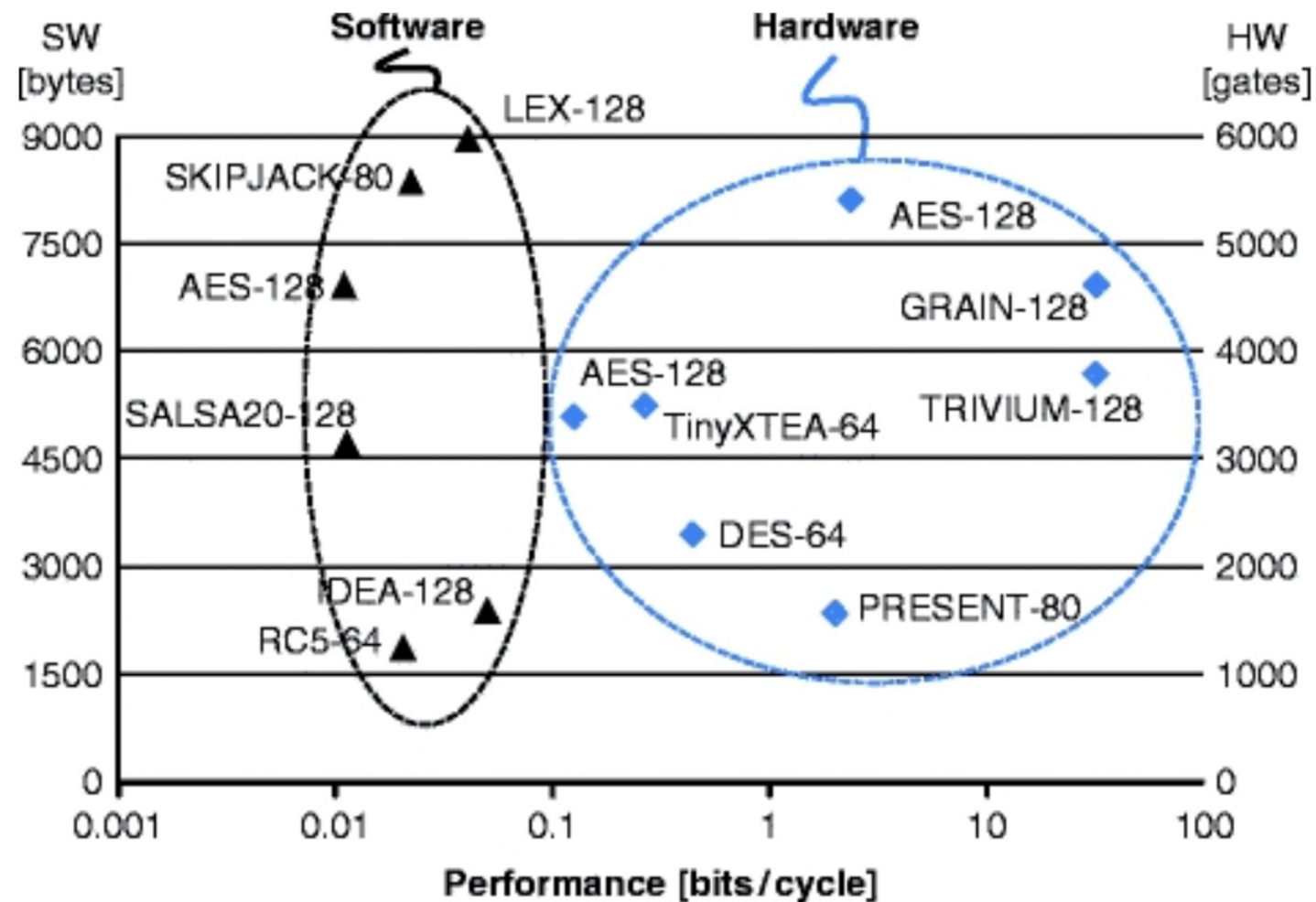
What do we learn here?



Schaumont, 2013

Cryptography on small embedded platforms

Hardware crypto-architectures have, on the average, a higher relative performance compared to embedded processors.

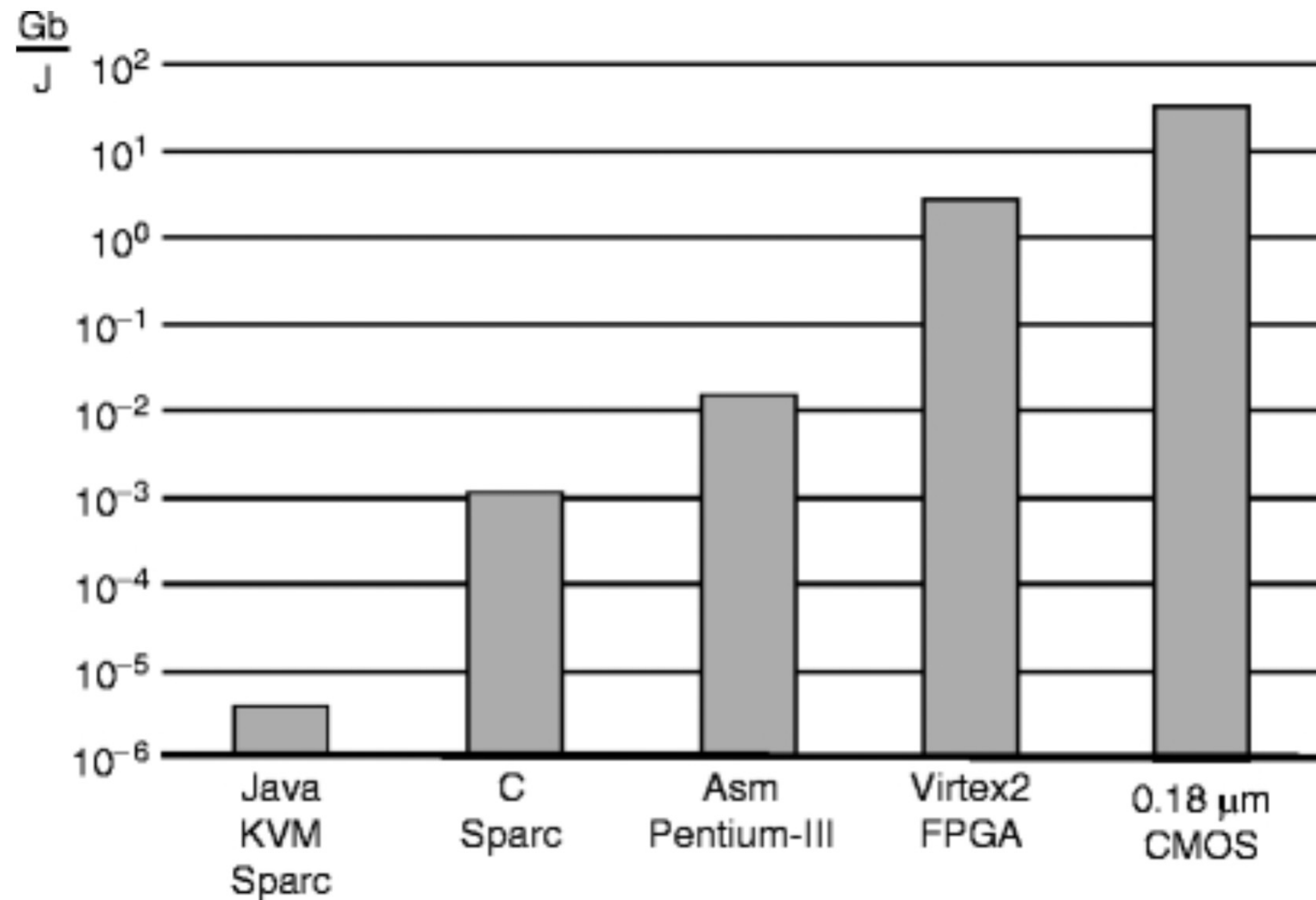


Schaumont, 2013

Cryptography on small embedded platforms

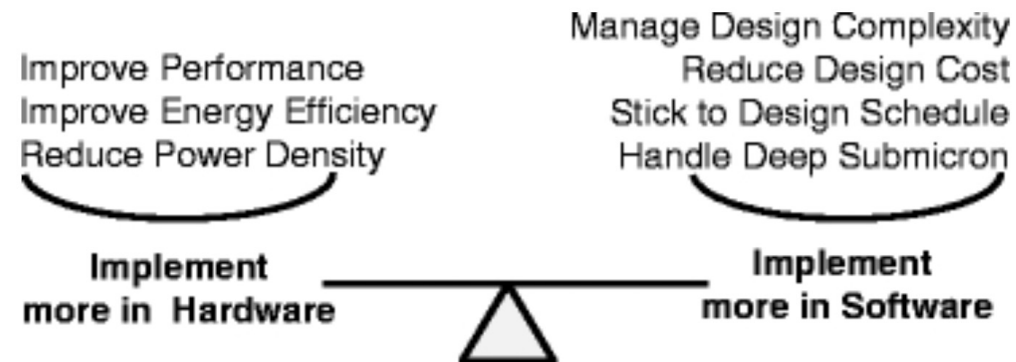
AES for
different
target
platforms

What do we
learn here?



Schaumont, 2013

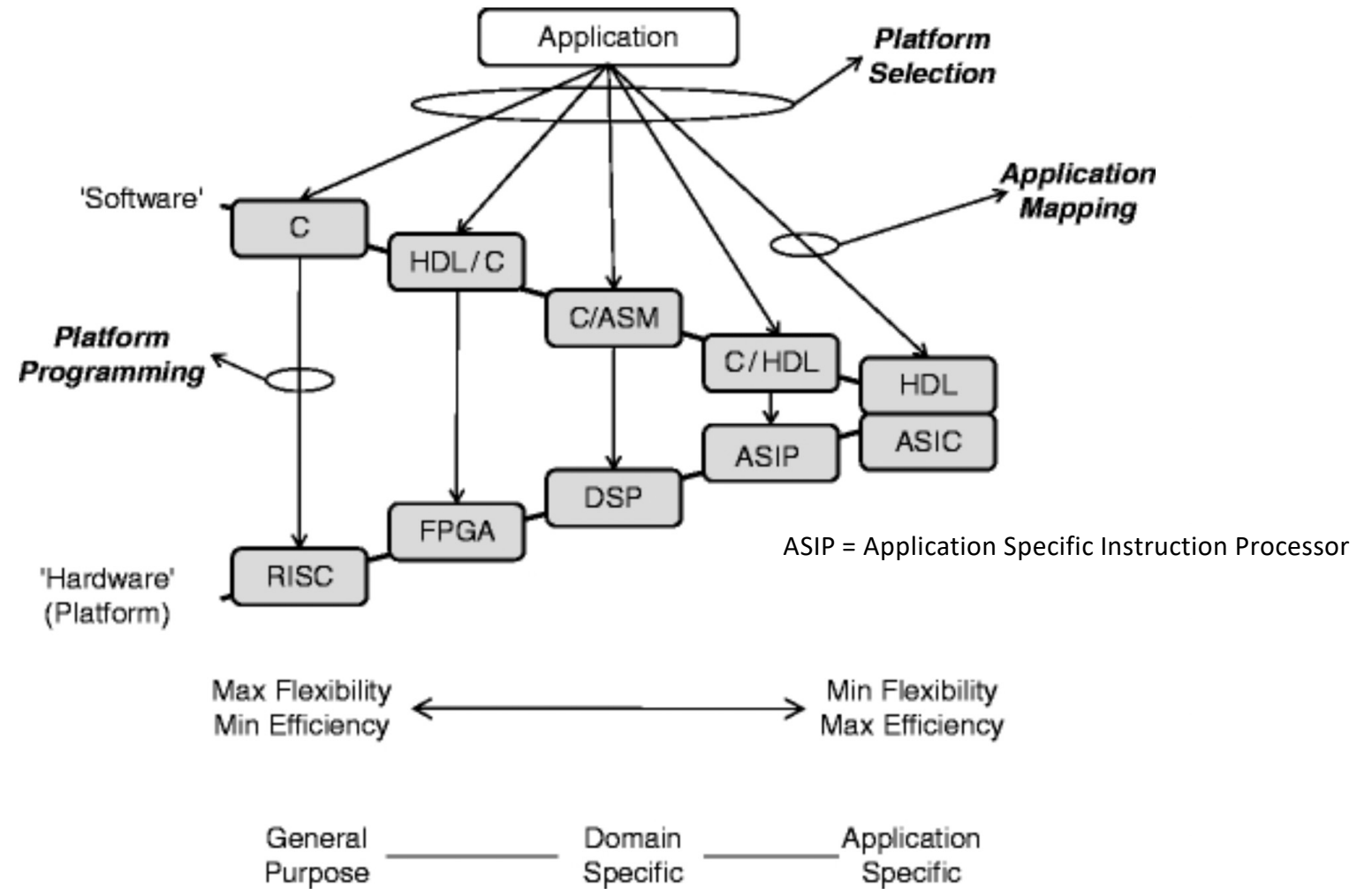
Driving factors in HW/SW co-design + dualism



	Hardware	Software
Design paradigm	Decomposition in space	Decomposition in time
Resource cost	Area (# of gates)	Time (# of instructions)
Constrained by	Time (clock cycle period)	Area (CPU instruction set)
Flexibility	Must be designed-in	Implicit
Parallelism	Implicit	Must be designed-in
Modeling	Model $\neq$ Implementation	Model $\sim$ Implementation
Reuse	Uncommon	Common



The HW/SW co-design space

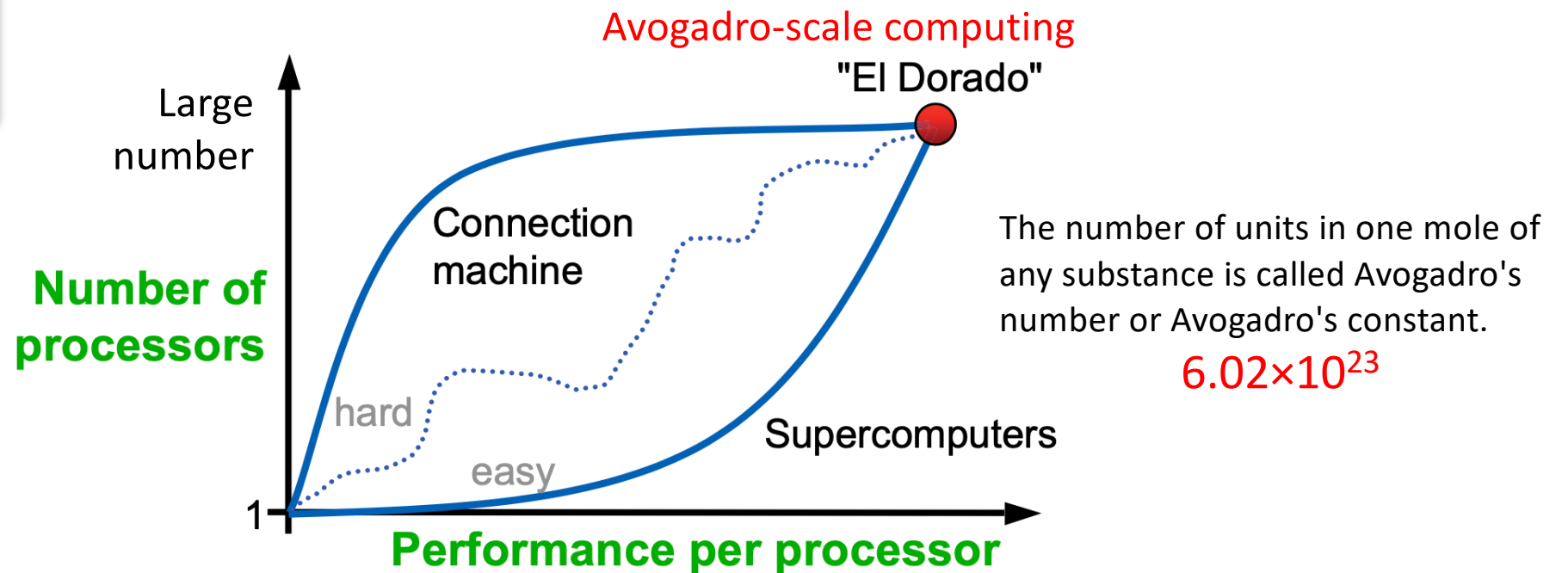


Schaumont, 2013

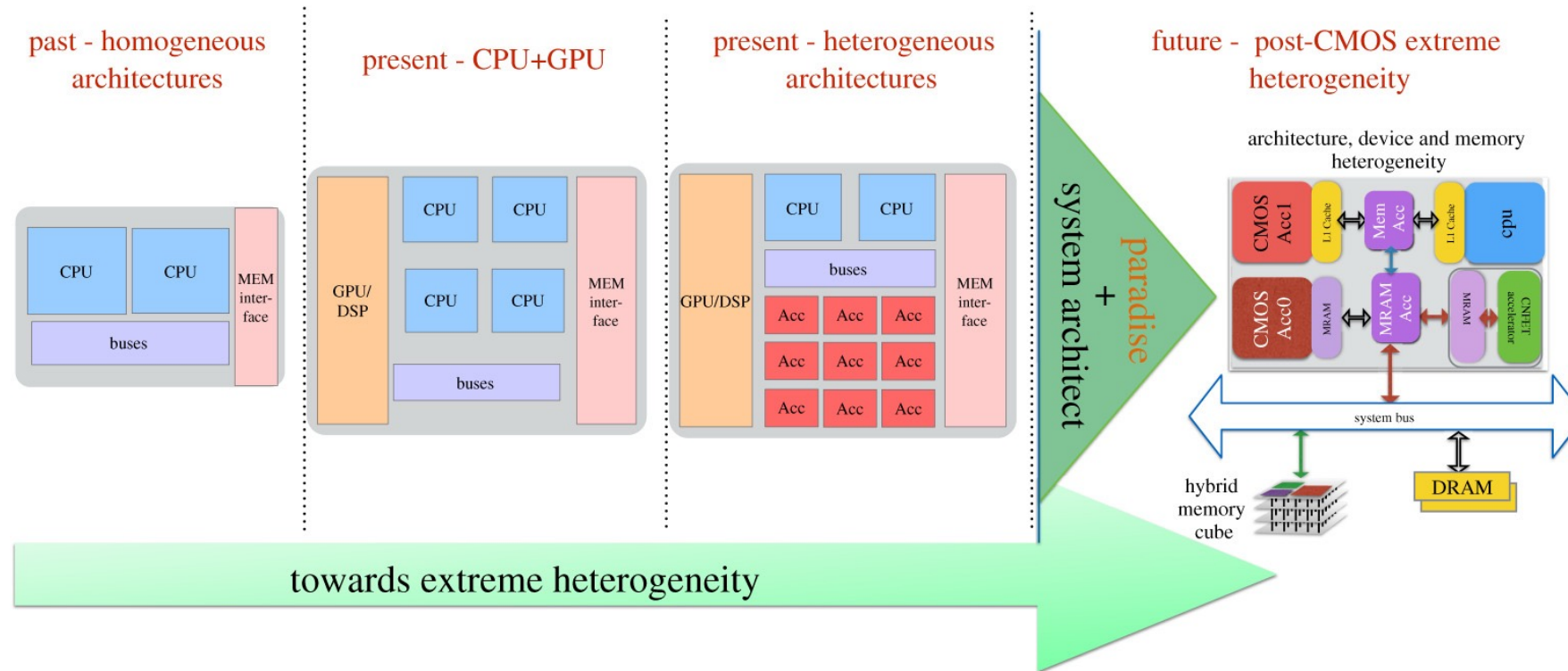
How do we get to the computing “El Dorado?”



CM 3: 900,000 AI-optimized, fully programmable cores. Dataflow architecture.

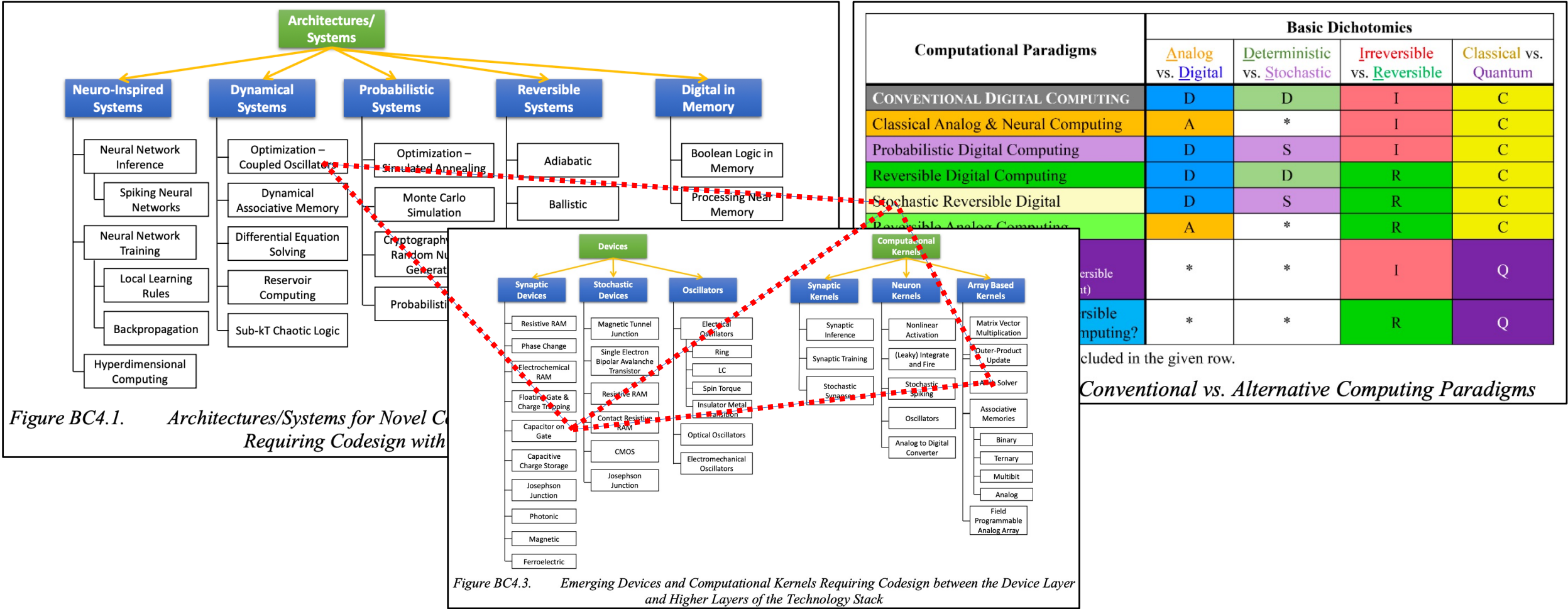


The goals and drivers have changed



<https://royalsocietypublishing.org/doi/10.1098/rsta.2019.0061>

Deep co-design across the entire stack





Christof Teuscher

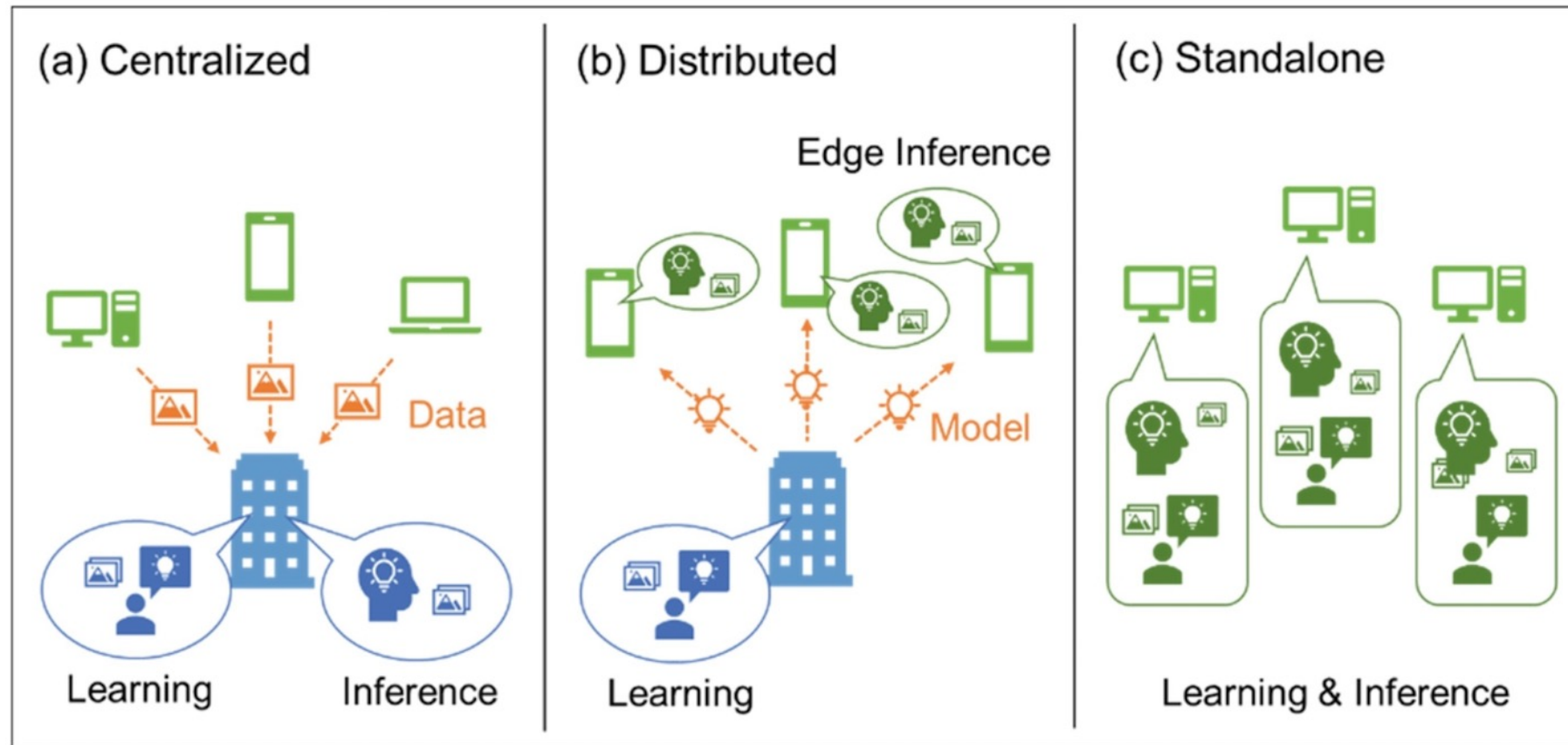
teuscher@pdx.edu

www.teuscher-lab.com/teaching



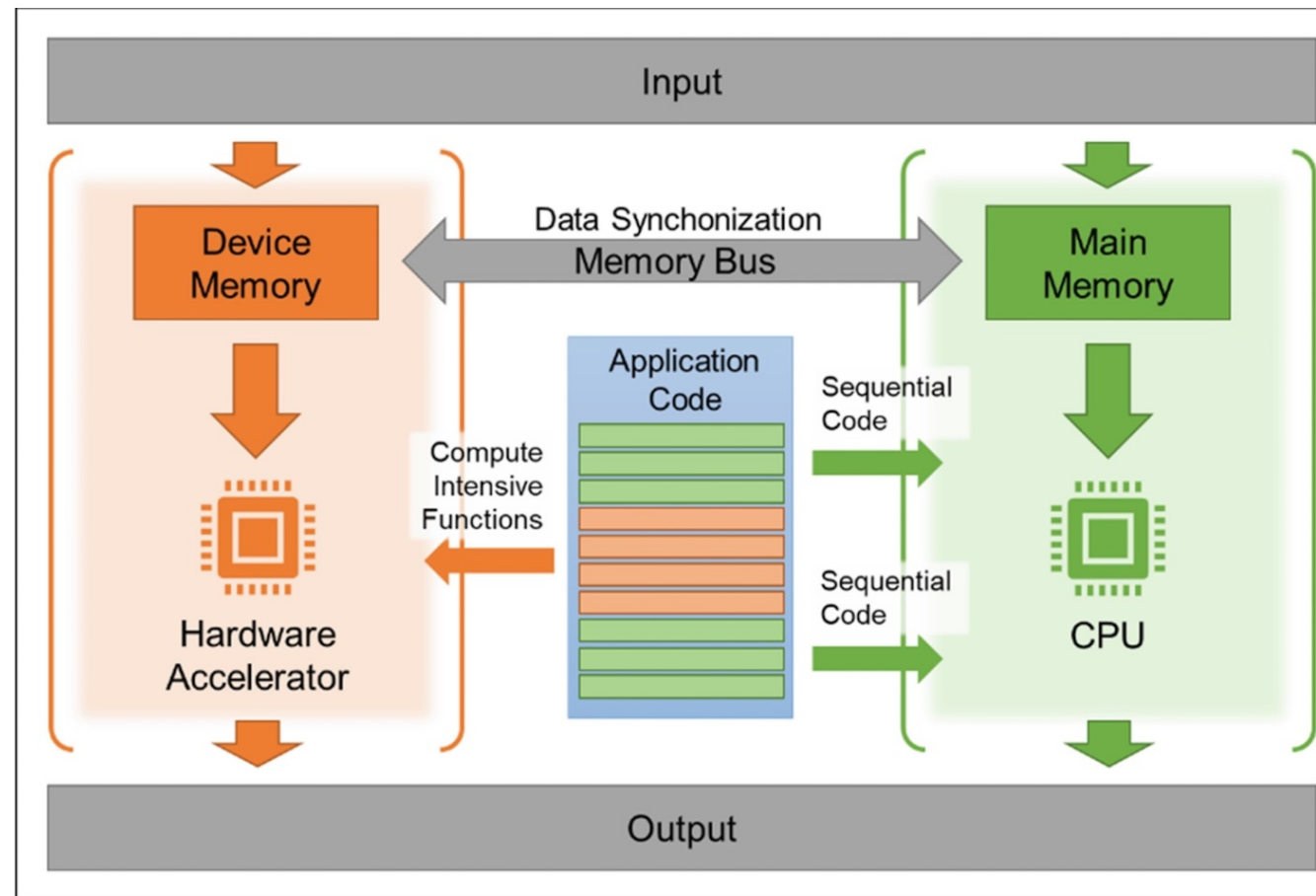
Hardware for AI/ML

AI HW accelerators: where can they be?



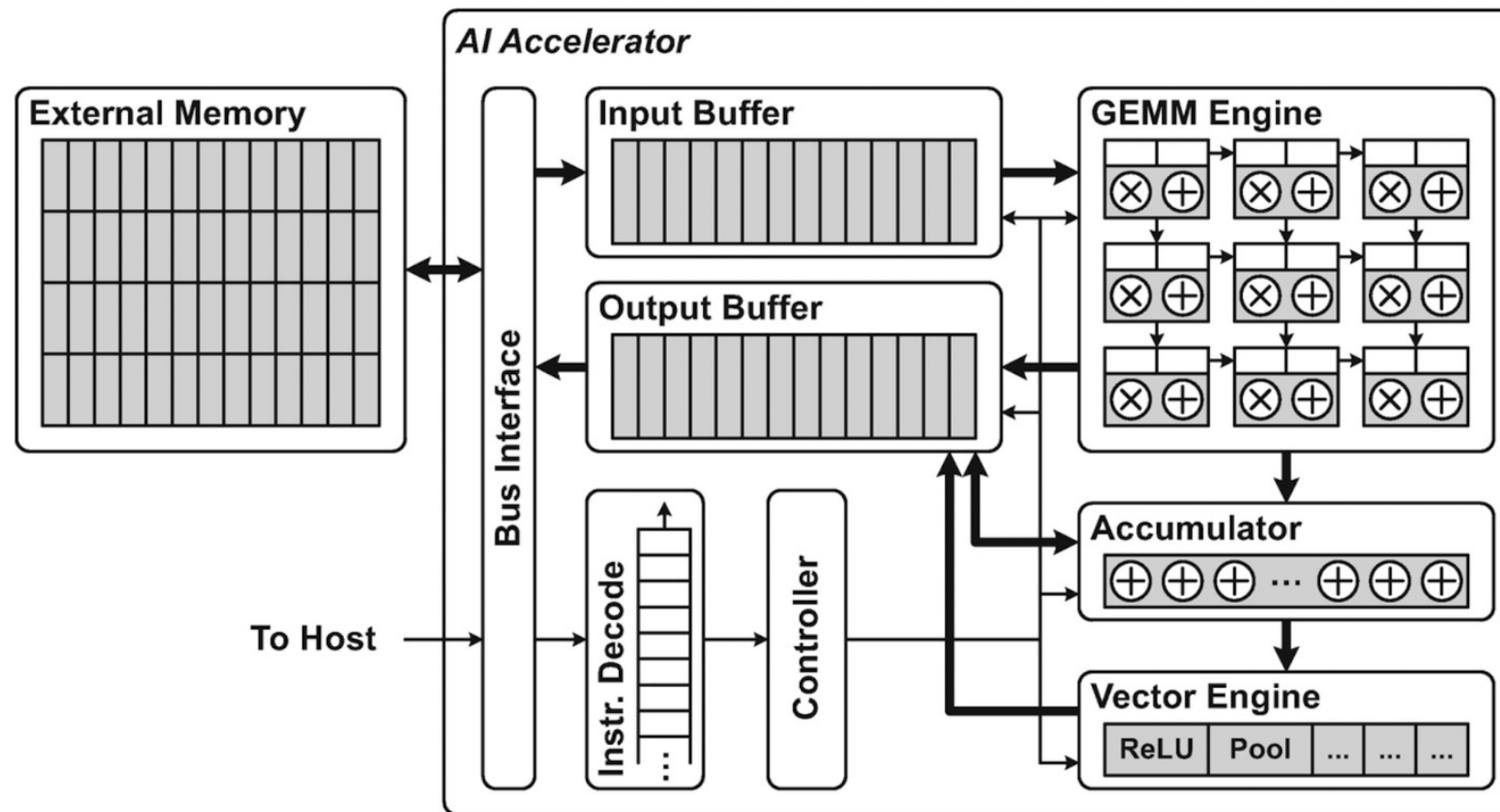
Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

General co-processing mechanism



Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

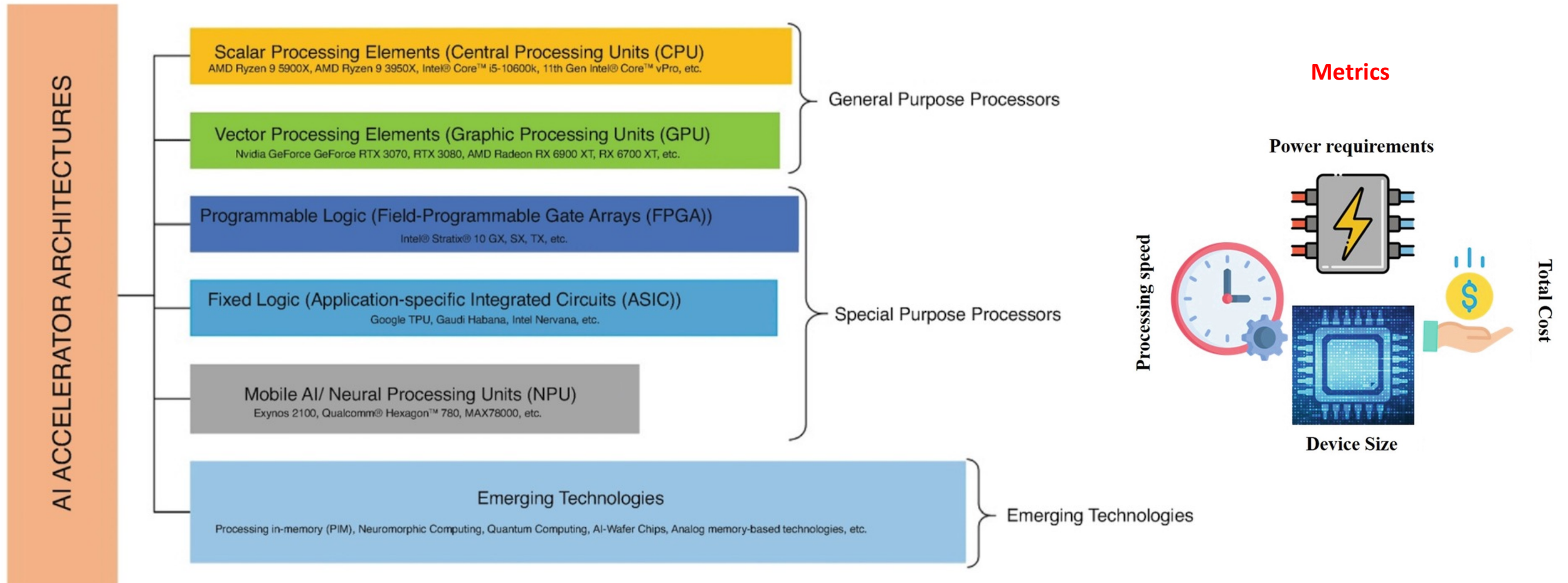
General AI accelerator architecture



GEMM = General Matrix-Matrix Multiply

Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

Hardware for AI/ML



Hardware for AI/ML: Trading time for space

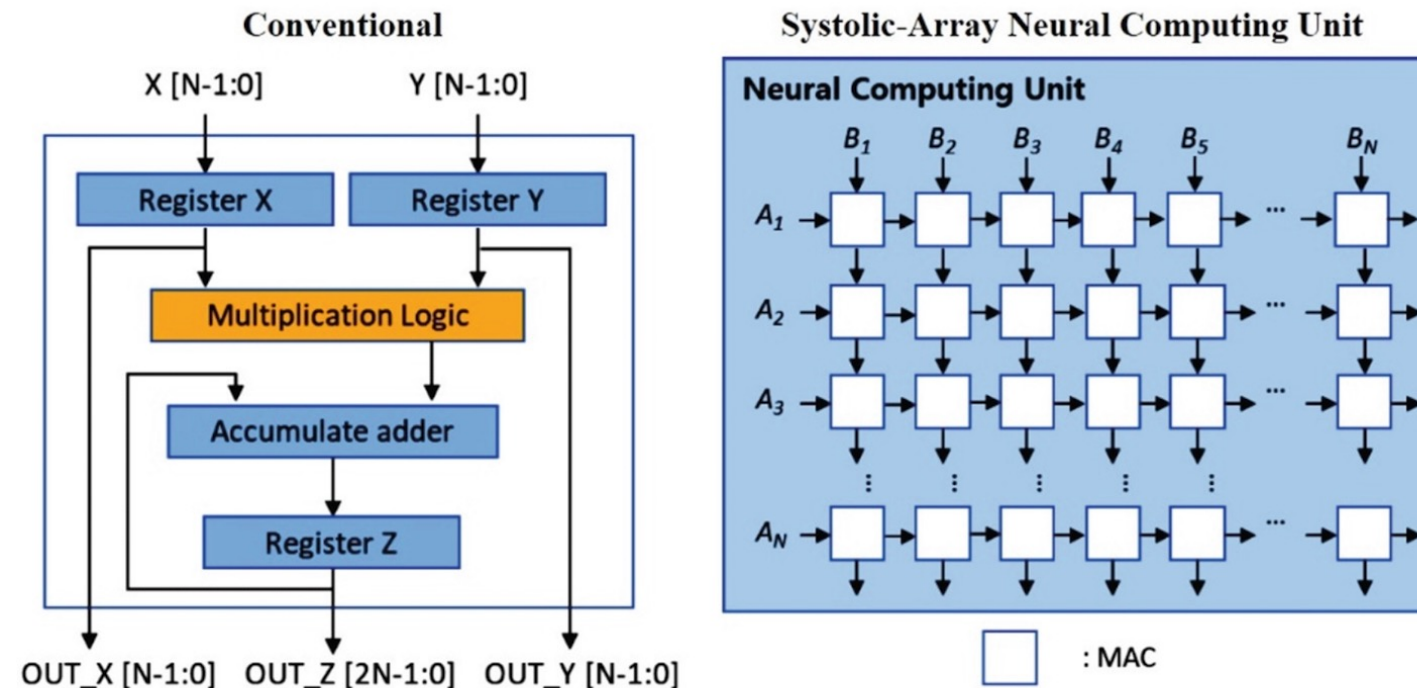


Fig. 19 A comparison of MAC operations performed on conventional and neural computing units.

Hardware for AI/ML

Processor	Power Consumption	Strengths	Limitations
CPU			
GPU			
FPGA			
ASIC			
Vision Processing Unit (VPU)			
Tensor Processing Unit (TPU)			



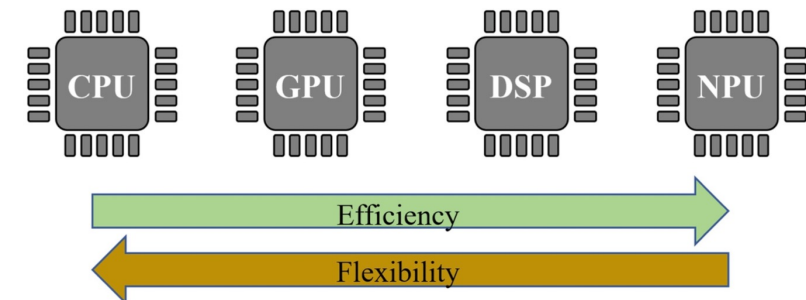
- Domain programmable
- Partially general-purpose
- Parallel workloads
- Power-hungry



- Hardware programmable
- Partially general-purpose
- Any workloads
- High performance
- Low power

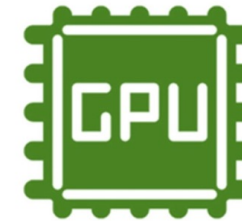


- Hard wired
- Special-purpose
- Dedicated workloads
- High performance
- Low power



Hardware for AI/ML

Processor	Power Consumption	Strengths	Limitations
CPU	High	→ Flexible → General-purpose processing → Complex instructions and tasks → System management	→ Possible memory access bottlenecks → Few cores (4–16)
GPU	High	→ Parallel cores (~1000 s of cores) → High-performance AI processing	→ Power consumption → Large footprint
FPGA	Medium	→ Configurable logic gates → Flexible → In-field re-programmability	→ Programming complexity
ASIC	Low	→ Custom logic designed with libraries → Faster processing → Small footprint	→ Fixed function → Expensive custom design
Vision Processing Unit (VPU)	Ultra-low	→ Dedicated image and vision co-processor → Small footprint	→ Limited dataset and batch size → Limited network support
Tensor Processing Unit (TPU)	Low to medium	→ Specialized tool support → Optimized for Tensor-Flow	→ Proprietary design → Limited framework support



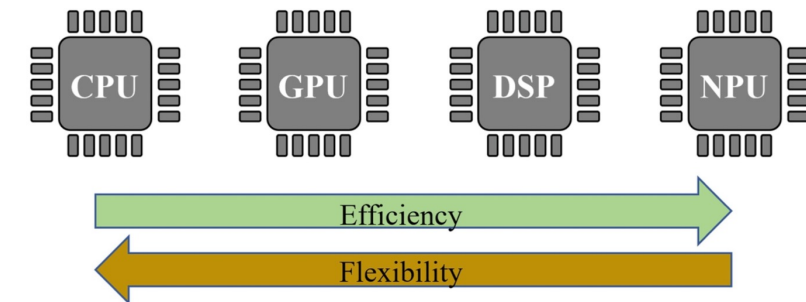
- Domain programmable
- Partially general-purpose
- Parallel workloads
- Power-hungry



- Hardware programmable
- Partially general-purpose
- Any workloads
- High performance
- Low power



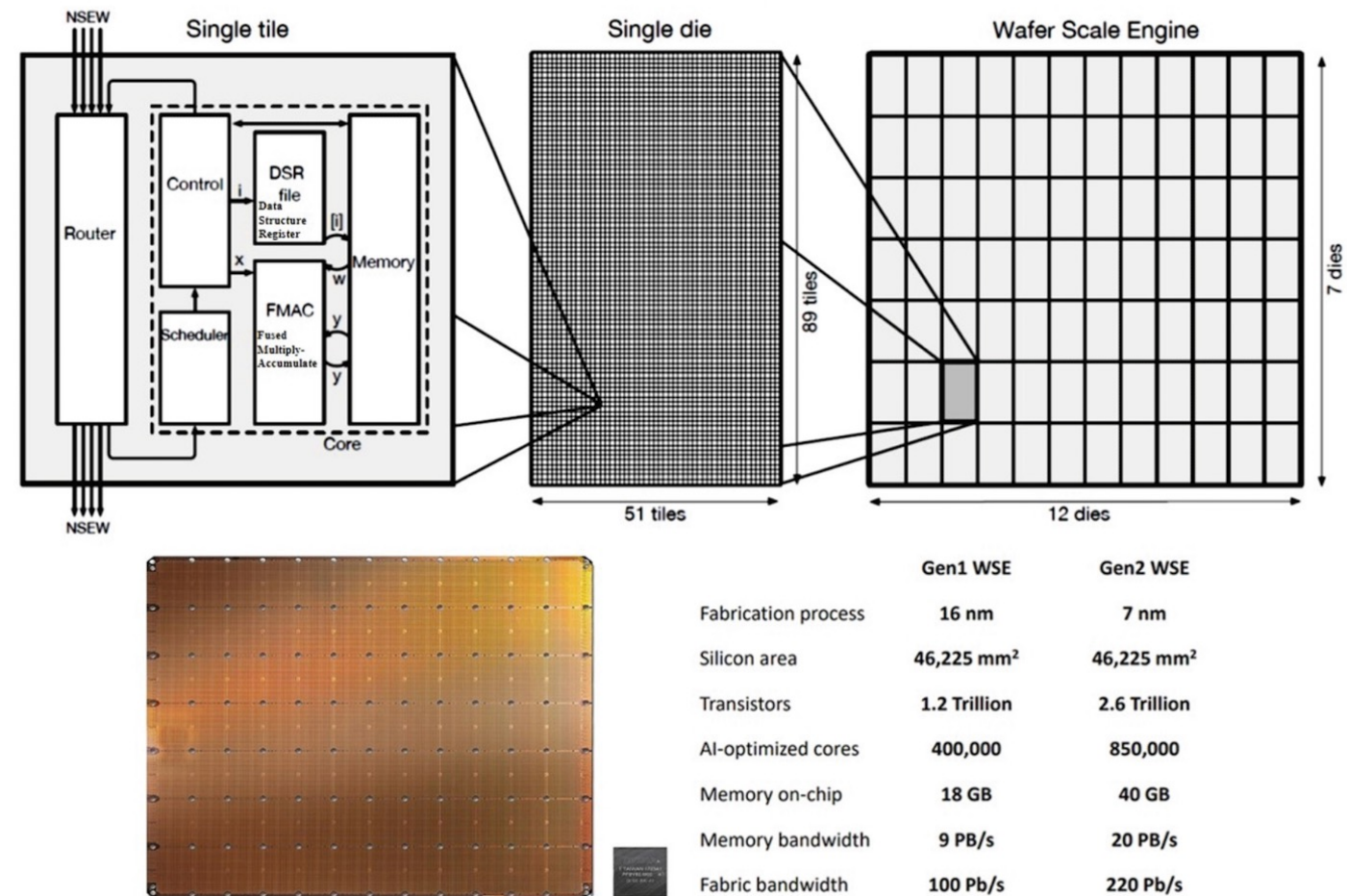
- Hard wired
- Special-purpose
- Dedicated workloads
- High performance
- Low power



Hardware for AI/ML

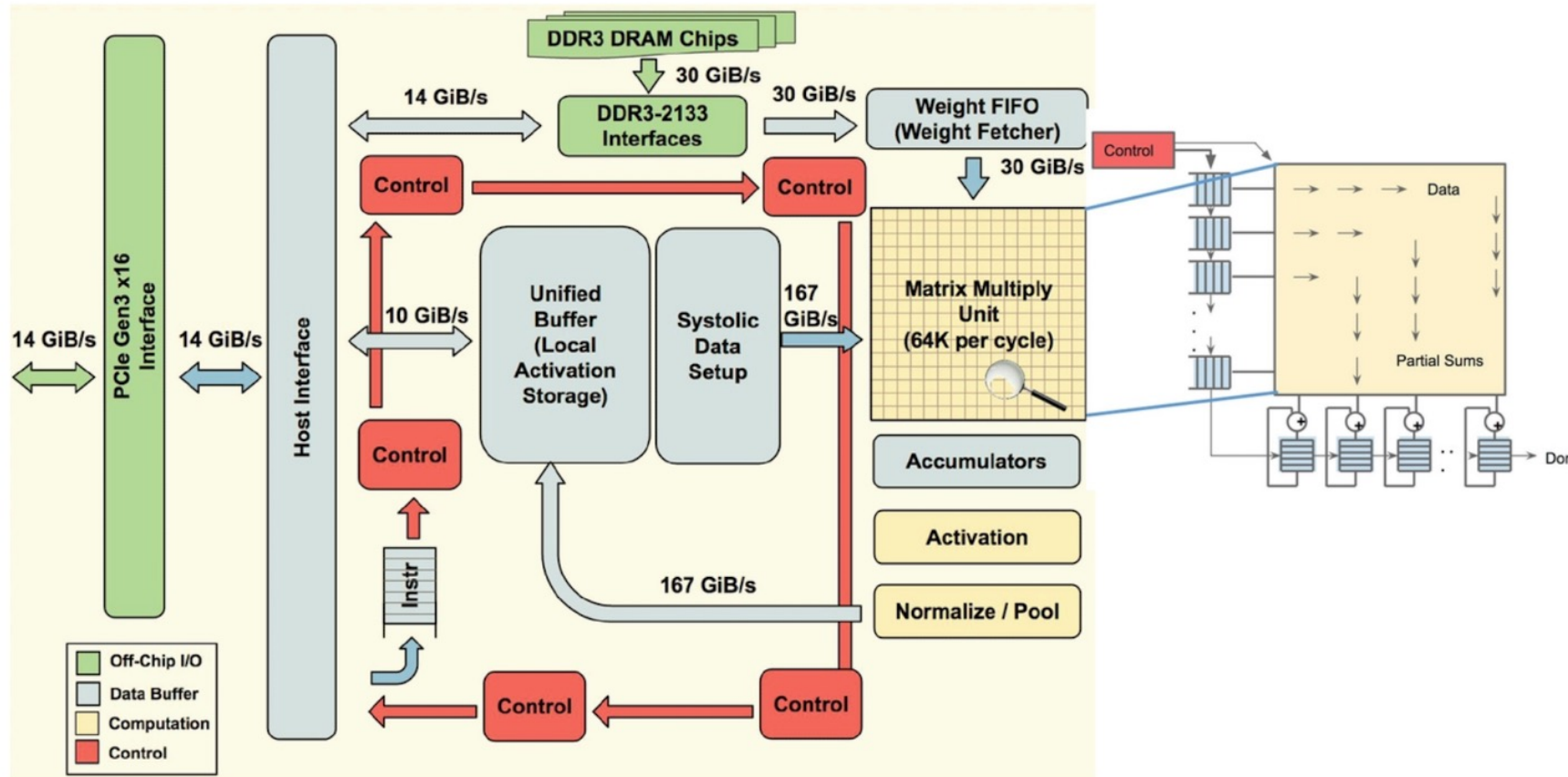
- A **tensor** is a mathematical object that generalizes scalars, vectors, and matrices to higher dimensions, often represented as a multi-dimensional array of numbers.
- Data Structure Registers (**DSRs**) are physical registers that are used to store **DSD** values.
- **DSDs** are a compact representation of a chunk of memory (e.g., tensor).
- **FMAC** is a specialized arithmetic unit designed to perform high-performance tensor operations for deep learning, specifically calculating $(a * b) + c$ in a single hardware step.

Cerebras Wafer Scale Engine (WSE)



Mishra et al., 2023

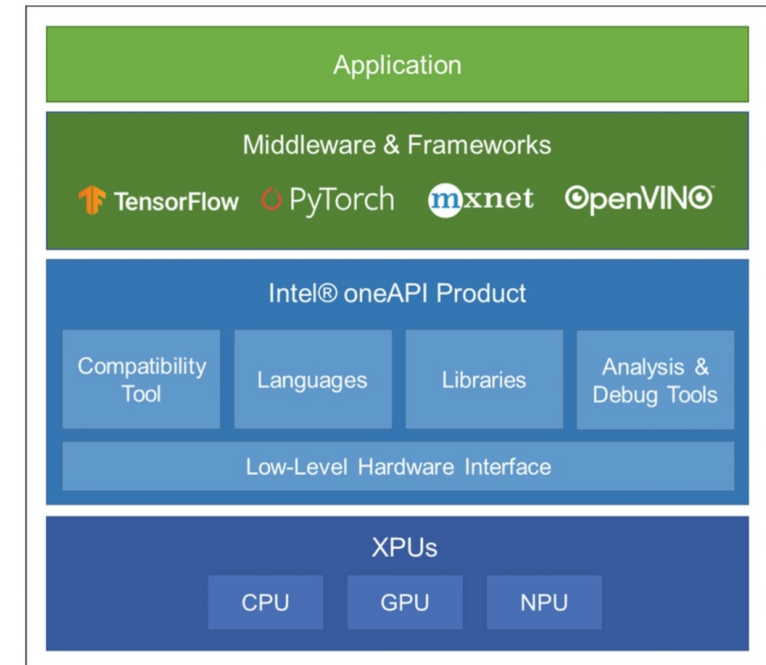
Architecture of a TPU accelerator



- The main component of TPU is the MXU that consists of 256 by 256 (i.e., 65,536) MACs to perform 8-bit mathematical operations.
- The MXU gets its input from the weighted FIFO and unified buffer (UB).

Deep learning frameworks

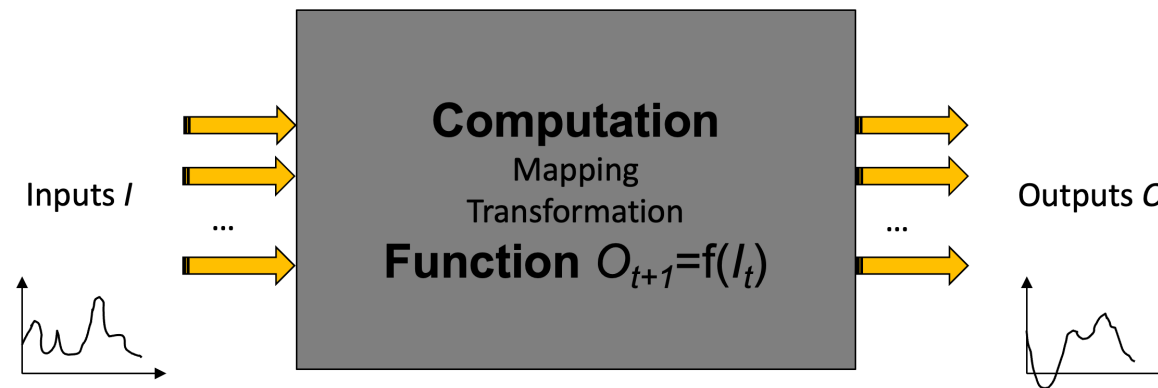
1. Deep learning frameworks provide interfaces to implement AI functionalities regardless of hardware or computing platforms.
2. Most deep learning frameworks also optimize the network in a **hardware-friendly manner** and utilize the hardware resources and acceleration functions as much as possible.
3. Because GPUs are commonly used in deep learning, deep learning frameworks **widely support constructing and deploying neural networks on GPU** and actively reflect the acceleration support from GPU vendors.
4. In addition, they provide optimizations on standalone accelerators such as **TPU** and specific environments such as cloud servers.



Mishra, Ashutosh; Cha, Jaekwang; Park, Hyunbin; Kim, Shiho. Artificial Intelligence and Hardware Accelerators, 2025

What is a (compute) kernel?

- In this context, a (compute) **kernel** is a **single computational routine that runs on a processor** (or GPU or some other HW).
- It is the unit of work you submit to the hardware: a self-contained function that takes some inputs, performs a defined computation, and produces outputs.



- **Examples:** matrix multiplication, a vector addition, a convolution operation, or a softmax. When you run a neural network, it is broken down into a sequence of kernel launches, each handling one operation.
- Different from “operating system kernel” and “kernel function in SVMs or Gaussian processes”

Arithmetic Intensity (AI)

- **Arithmetic Intensity (AI)** measures how much computation a program performs relative to how much data it moves. It is defined as the ratio of floating-point operations (FLOPs) to bytes transferred to and from memory:

$$I = \frac{\text{FLOPs}}{\text{Bytes transferred}}$$

AI < 1: moves more data than it computes.

AI > 1: computation larger than the data movement.

- The unit is **FLOPs per byte** (or FLOP/B).
- The purpose is to characterize whether a workload is **bottlenecked by compute throughput or by memory bandwidth**.
- A workload with low AI moves a lot of data relative to the work done.
- A workload with high AI does a lot of computation per byte fetched.
- This distinction matters because modern processors have a much higher ratio of peak compute to peak memory bandwidth.
- **AI says nothing about bottlenecks!**

Arithmetic intensity example (1)

- **A simple vector addition**
 - $C[i] = A[i] + B[i]$, reads two arrays and writes one.
 - For N elements, that is $3N \times 4$ bytes transferred and N additions.
 - The AI is roughly 1 FLOP / 12 bytes, or about **0.083 FLOP/B**.
 - This is extremely low.

Arithmetic intensity example (1)

- General Matrix Multiply (GEMM)**

$$C = \alpha AB + \beta C$$

$\alpha = 1$ and $\beta = 0$ it reduces to a plain matrix multiply $C = AB$.

- A dense matrix-matrix multiplication of two $N \times N$ matrices performs roughly $2N^3$ FLOPs and transfers about $3N^2 \times 4$ bytes (the three matrices).

$$I = \frac{2N^3}{3N^2 \times 4}$$

A				B				C		
2	1	0	×	1	3	2	=	2	7	8
3	0	2		0	1	4		7	9	8
1	2	1		2	0	1		3	5	11

Can you explain these values?

- Dense means every element of the matrix is stored and participated in the computation. There are no zeros that are skipped or compressed away.

Arithmetic intensity example (2)

- General Matrix Multiply (GEMM)**

$$C = \alpha AB + \beta C$$

$\alpha = 1$ and $\beta = 0$ it reduces to a plain matrix multiply $C = AB$.

- A dense matrix-matrix multiplication of two $N \times N$ matrices performs roughly $2N^3$ FLOPs and transfers about $3N^2 \times 4$ bytes (the three matrices).

$$I = \frac{2N^3}{3N^2 \times 4}$$

A				B				C		
2	1	0	×	1	3	2	=	2	7	8
3	0	2		0	1	4		7	9	8
1	2	1		2	0	1		3	5	11

- Each output cell $C[i][j]$ requires N multiplications and $N-1$ additions, which is roughly $2N$ operations. There are N^2 output cells (an $N \times N$ matrix). So total FLOPs = $2N \times N^2 = 2N^3$.
- There are three matrices: A, B, and C. Each is $N \times N$ and stores 32-bit floats, so each matrix occupies $N^2 \times 4$ bytes. Reading A and B and writing C gives $3 \times N^2 \times 4$ bytes total.

- Dense means every element of the matrix is stored and participated in the computation. There are no zeros that are skipped or compressed away.

Arithmetic intensity example (3)

- **General Matrix Multiply (GEMM)**

$$C = \alpha AB + \beta C$$

$\alpha = 1$ and $\beta = 0$ it reduces to a plain matrix multiply $C = AB$.

- What happens for large N?

$$I = \frac{2N^3}{3N^2 \times 4}$$

Arithmetic intensity example (4)

- **General Matrix Multiply (GEMM)**

$$C = \alpha AB + \beta C$$

$\alpha = 1$ and $\beta = 0$ it reduces to a plain matrix multiply $C = AB$.

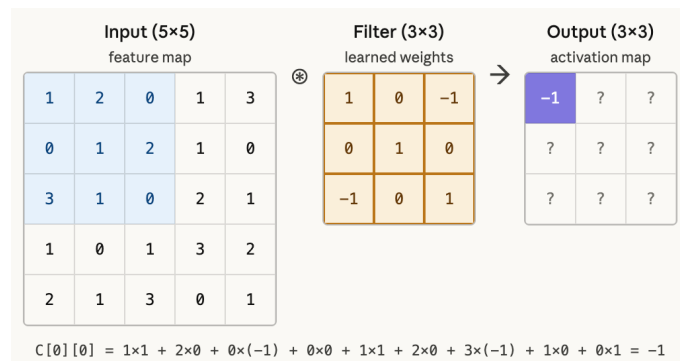
- What happens for large N?

$$I = \frac{2N^3}{3N^2 \times 4} = \frac{2N}{12} = \frac{N}{6}$$

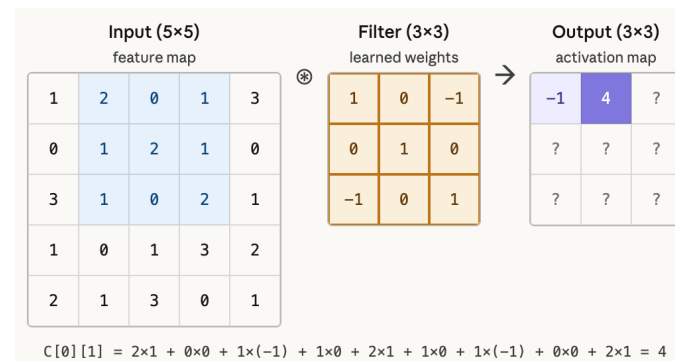
- For large N, AI grows as $N/6$, which can reach hundreds of FLOP/B for large matrices. This is compute-bound.
 - Arithmetic intensity grows linearly with N.
 - A 100×100 matrix multiply has $I \approx 17$ FLOP/B.
 - **GEMM is one of the few workloads that actually saturates GPU compute units, and why so much of deep learning is engineered to reduce to matrix multiplication.**

Arithmetic intensity example (5)

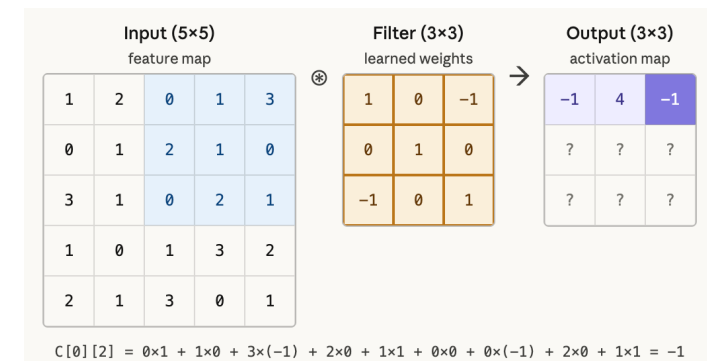
- **A convolutional layer** in a neural network.
 - For a typical image convolution with a 3×3 kernel, each output value requires 9 multiply-accumulate operations (18 FLOPs),
 - But the weight reuse across spatial positions raises AI to perhaps 10-100 FLOP/B depending on batch size and layer dimensions.



Step 1



Step 2



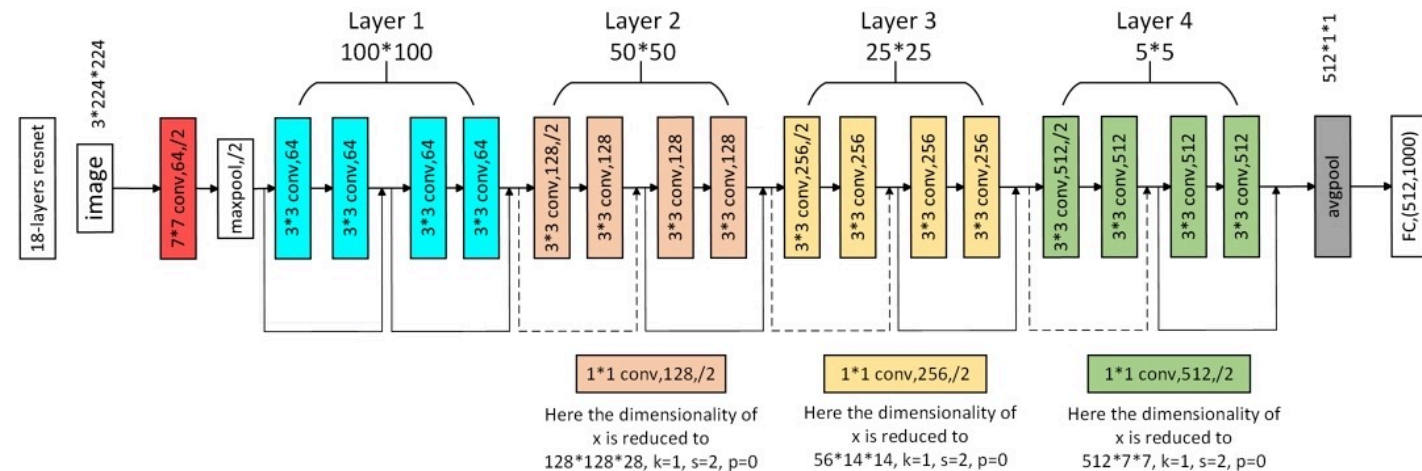
Step 3

Arithmetic intensity example (6)

- ResNet-18: torchinfo report

```

=====
Total params: 11,689,512
Trainable params: 11,689,512
Non-trainable params: 0
Total mult-adds (G): 1.81
=====
Input size (MB): 0.60
Forward/backward pass size (MB): 39.75
Params size (MB): 46.76
Estimated Total Size (MB): 87.11
=====
  
```



Arithmetic intensity example (7)

- **ResNet-18: torchinfo report**

```
=====
Total params: 11,689,512
Trainable params: 11,689,512
Non-trainable params: 0
Total mult-adds (G): 1.81
=====
```

```
Input size (MB): 0.60
Forward/backward pass size (MB): 39.75
Params size (MB): 46.76
Estimated Total Size (MB): 87.11
=====
```

Using your formula with a batch size of 1 (as shown here):

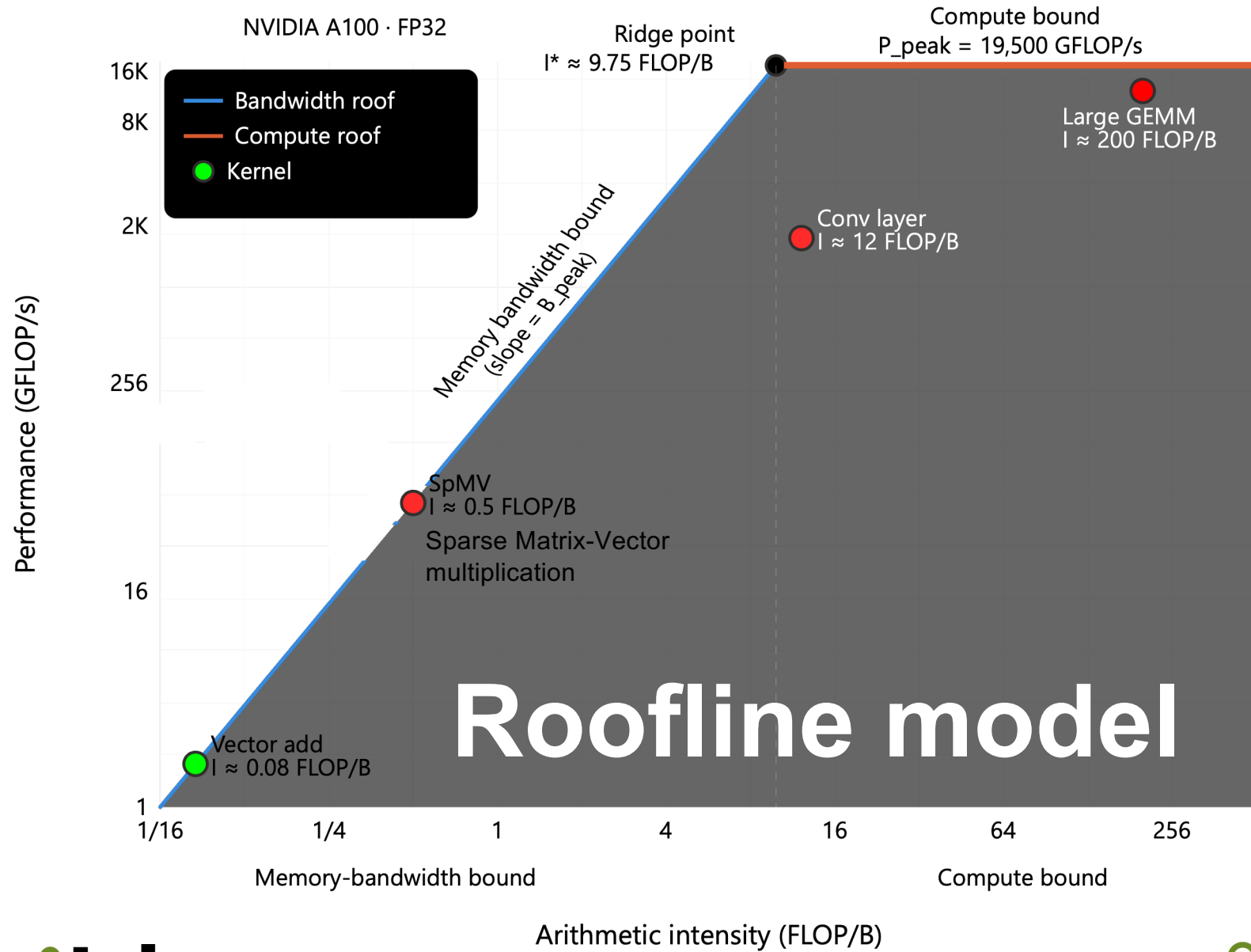
- Numerator: $2 \times 1.81\text{G MACs} = 3.62\text{G operations}$
- Denominator: $(46.76\text{ MB weights} + 39.75\text{ MB activations}) = 86.51\text{ MB} = 0.08651\text{ GB}$
- Intensity: $3.62\text{G} / 0.08651\text{G bytes} = 41.8\text{ operations per byte}$

- A single forward pass requires **1.81 billion** multiply-accumulate operations. For training (forward and backward), you'd typically use roughly 2-3x this value, depending on how backward pass computation is counted.
- Total params (**11.69M**) represents all the weights that need to be stored and updated during training. Convert this to bytes by assuming 4 bytes per parameter (for float32): **46.76 MB**.
- Input size (**0.60 MB**) is just the single input image. Forward/backward pass size (**39.75 MB**) is the activation memory needed to store intermediate tensors during both passes. The total estimated size (**87.11 MB**) is the full memory footprint if you load weights plus all intermediate activations.

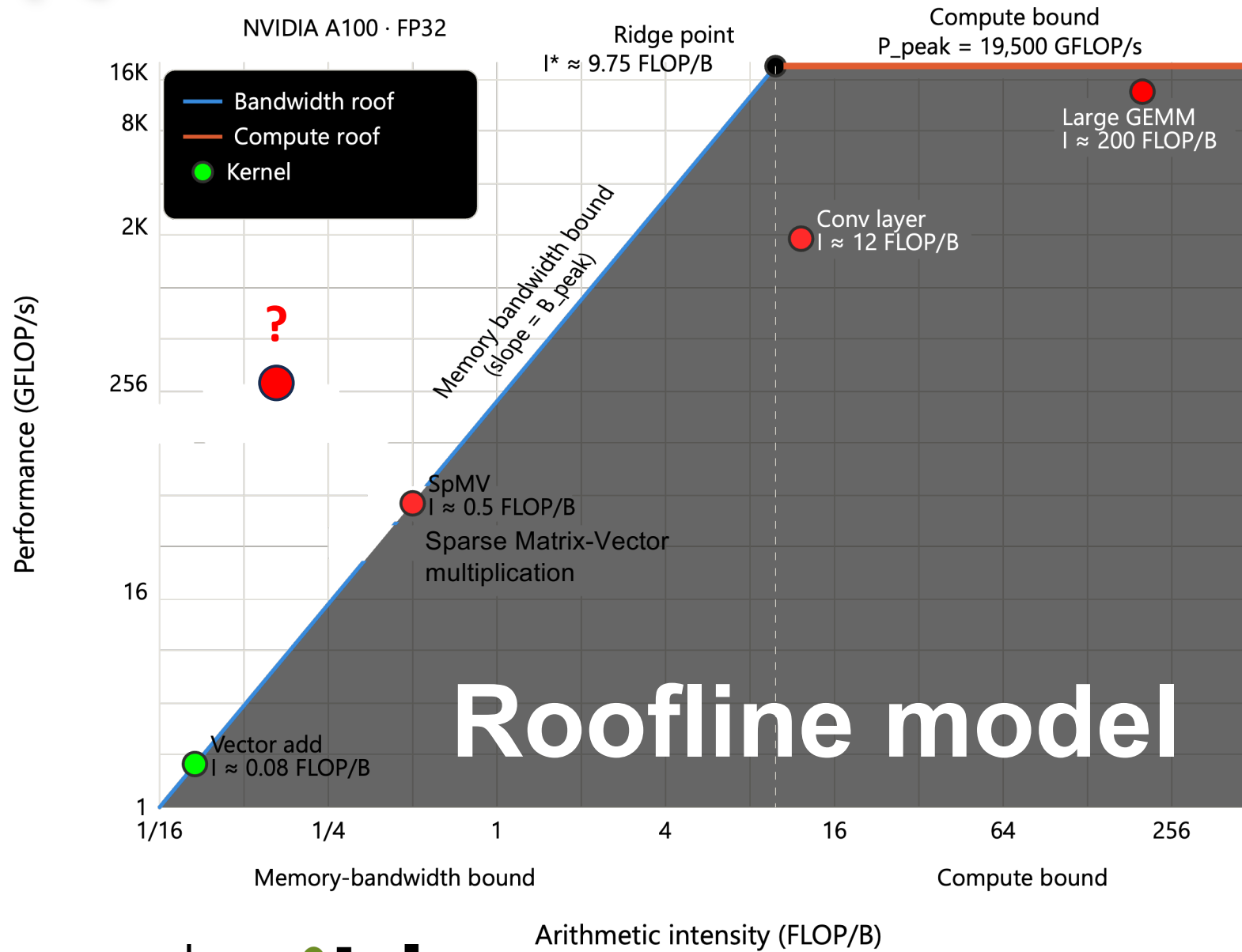
The roofline model

- Introduced by Williams, S., Waterman, A., & Patterson, D. (2009). **Roofline: An insightful visual performance model for multicore architectures.** Communications of the ACM, 52(4), 65-76.
<https://doi.org/10.1145/1498765.1498785>
- A visual and analytical framework for **predicting the achievable performance of a kernel on a given hardware platform**. It combines two hardware limits into one plot:
 - **The peak compute throughput P_{peak}** (in GFLOP/s) is a horizontal ceiling: no kernel can exceed this regardless of how compute-efficient it is.
 - **The peak memory bandwidth B_{peak}** (in GB/s) defines a diagonal ceiling: for a kernel with arithmetic intensity I , the maximum achievable performance is $I \times B_{\text{peak}}$. This rises linearly with AI because more computation per byte means memory bandwidth is used more productively.
- Together, these two bounds form a "roof" shape. The **ridge point** is the AI value where the two bounds are equal:

$$I^* = \frac{P_{\text{peak}}}{B_{\text{peak}}}$$



What can we
learn from this
model to build
efficient HW?



- A kernel that achieves the roofline bound is one that fully saturates the relevant resource.
- **Kernels that fall below the roofline are inefficient relative to the hardware limit.**
- A point above the roofline would mean the kernel exceeds either the memory bandwidth bound or the compute bound, both of which are physical limits of the hardware. It almost always means the arithmetic intensity was undercounted.

What to do if compute-bound below roofline?

The causes and remedies depend on what is preventing full utilization. GPU perspective:

1. **Instruction-level parallelism.** Long dependency chains, where each operation must wait for the result of the previous one, serialize execution and leave functional units idle. Restructuring the computation to expose more independent operations, or unrolling loops, can help.
2. **Occupancy.** GPUs hide latency by switching between warps. If too few warps are active simultaneously (due to high register usage or large shared memory allocation per thread block), the scheduler has nothing to switch to and stalls. Reducing register pressure or tuning thread block size to increase occupancy can close the gap.
3. **Instruction mix.** Not all FLOPs are equal. If the kernel mixes floating-point operations with integer indexing, memory address calculations, or branch instructions, the floating-point units are not busy the whole time. The roofline's P_{peak} assumes the hardware is issuing floating-point operations continuously, which is rarely true in practice.
4. **Precision mismatch.** If you draw the roofline using FP32 peak throughput but the kernel runs in FP64, or vice versa, the effective ceiling is different. Modern GPUs often have 2:1 or even 64:1 ratios between FP32 and FP64 throughput.
5. **Tensor core underutilization.** On GPUs with tensor cores (like the A100), the FP32 scalar peak and the tensor core peak are very different numbers. A kernel that does not tile its matrix operations to hit tensor core dimensions (multiples of 16) will use scalar CUDA cores instead and sit well below the tensor core roofline.

The practical response is to use a profiler to identify which specific bottleneck applies, rather than guessing. The roofline tells you there is a gap; the profiler tells you why.

What to do if memory-bound below roofline?

The causes and remedies depend on what is preventing full utilization. GPU perspective:

- **Memory access pattern.** The most common cause. GPUs achieve peak bandwidth only when threads in a warp access consecutive memory addresses simultaneously, called coalesced access. If threads scatter across memory, the hardware issues many small transactions instead of one wide one, and effective bandwidth collapses.
- **Cache thrashing.** If the working set is just slightly larger than a cache level, data gets evicted and re-fetched repeatedly. The bytes transferred to DRAM are much higher than the algorithm's minimum, which also means the measured AI is lower than the theoretical AI.
- **Bandwidth from the wrong level.** The roofline bandwidth ceiling is usually drawn using peak DRAM bandwidth, but if your data fits in L2 or L1, you should be comparing against those ceilings instead. This is why multi-level roofline plots, as produced by Nsight Compute, are more informative than a single-ceiling version.
- **Insufficient memory-level parallelism.** DRAM controllers achieve peak bandwidth by issuing many in-flight requests simultaneously. If the kernel issues memory requests sequentially (because loop iterations are dependent, or because there are too few threads active) the pipeline stays empty and bandwidth is underutilized.
- **Atomic operations.** If many threads write to the same memory location using atomics, those operations serialize and become a bottleneck independent of bandwidth.
- **Small working set.** If the total data is small, there simply is not enough traffic to keep the memory system busy long enough to amortize startup latency. This is less a fixable inefficiency and more a fundamental limit of small problem sizes: latency dominates over bandwidth.

Food for thought

1. Why is executing a neural network on a general purpose computer inefficient?
2. What circuitry/processor would you design to find the shortest path in large graphs?
 - What if the graph is dense?
 - What if the graph is sparse?
3. What circuitry/processor would you design to recognize QR codes for an embedded camera?
 - Recognition needs to be fast and low-power.

